

Bases de Datos

Grado en Ciencia de Datos

Proyecto

Café Nesprisa

Autores:
Marta Pla Tudela



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

ÍNDICE

ENUNCIADO DEL PROBLEMA.....	3
1. TAREA 1: ANÁLISIS Y DISEÑO CONCEPTUAL	4
1.1 DIAGRAMA DE CLASES	4
1.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR.....	5
1.3 REQUISITOS DE PROCESO Y/O CONSULTA	5
2. TAREA 2: DISEÑO LÓGICO.....	6
2.1 ESQUEMA RELACIONAL	6
2.2 RESTRICCIONES DE INTEGRIDAD QUE NO SE HAN PODIDO EXPRESAR.....	8
2.3 NORMALIZACIÓN.....	9
2.4 DIRECTRICES DE RESTAURACIÓN DE LA INTEGRIDAD REFERENCIAL	16
2.5 TRANSACCIONES.....	18
3. TAREA 3: CREACIÓN Y CARGA DE LA BASE DE DATOS EN ORACLE.....	19
3.1 INSTRUCCIONES DE LA CREACIÓN DE LA BASE DE DATOS	19
3.1.1 Creación de la tabla CLIENTE	19
3.1.2 Creación de la tabla AGENDA	19
3.1.3 Creación de la tabla VARIEDAD_CAFÉ	19
3.1.4 Creación de la tabla OFERTA.....	20
3.1.5 Creación de la tabla COMPRA.....	20
3.1.6 Creación de la tabla COMPRA_VARIEDAD	20
3.1.7 Creación de la tabla ADITIVO	21
3.1.8 Creación de la tabla ARTIFICIAL	21
3.1.9 Creación de la tabla NATURAL.....	21
3.1.10 Creación de la tabla INCLUYE	21
3.1.11 Creación de la tabla GRANO	22
3.1.12 Creación de la tabla PAÍS	22
3.1.13 Creación de la tabla REGIÓN	22
3.1.14 Creación de la tabla SE_PRODUCE	22
3.1.15 Creación de la tabla MEZCLA.....	23
3.1.16 Creación de la tabla VARIEDAD_ESPECIAL	23
3.2 DIAGRAMA GENERADO POR ORACLE CON EL DATA MODELER	24
3.3 CARGA DE LA BASE DE DATOS	24
3.3.1 Funciones de carga	25
3.3.2 Secuencias	31
3.3.3 Carga de la tabla CLIENTE	32
3.3.4 Carga de la tabla AGENDA.....	32
3.3.5 Carga de la tabla VARIEDAD_CAFÉ	33
3.3.6 Carga de la tabla OFERTA	34
3.3.7 Carga de la tabla REGIÓN.....	35
3.3.8 Carga de la tabla SE_PRODUCE	36
3.3.9 Carga de la tabla MEZCLA	37
3.3.10 Carga de la tabla VARIEDAD_ESPECIAL	39
3.3.11 Carga de la tabla PAÍS.....	40
3.3.12 Carga de la tabla GRANO	40
3.3.13 Carga de la tabla NATURAL	41
3.3.14 Carga de la tabla INCLUYE	41
3.3.15 Carga de la tabla ADITIVO	42
3.3.16 Carga de la tabla ARTIFICIAL.....	43

3.3.17 Carga de la tabla COMPRA	44
3.3.18 Carga de la tabla COMPRA_VARIEDAD.....	44
3.3.19 Recalcular el importe de las compras	45
4. TAREA 4: INFORMES Y PROGRAMACIÓN.....	46
4.1 INFORMES.....	46
4.1.1 Listado 1	47
4.1.2 Listado 2	48
4.1.3 Listado 3	49
4.1.4 Listado 4	50
4.1.5 Listado 5	51
4.1.6 Listado 6	52
4.2 PROGRAMACIÓN.....	53
4.2.1 Funciones	53
4.2.2 Procedimientos	54
4.2.3 Disparadores	56

ENUNCIADO DEL PROBLEMA

Un distribuidor de café de la marca Nesprisa ha decidido crear una base de datos para almacenar la información referente a la actividad comercial que debe realizar.

En la tienda tienen un almacén con las diferentes variedades de café de que dispone la marca, cada variedad de café se identifica por un código y tiene una denominación (única) (ristreto, intenso, etc.), el PVP de la caja de 10 unidades, la intensidad de sabor (1 a 12), se quiere saber si lleva cafeína, y, en el caso de que lleve cafeína, el nivel de cafeína (alto, medio, bajo).

A los clientes se les abre una ficha con el DNI (único), el nombre, uno o varios teléfonos y el email. Cuando los clientes realizan una compra se guardan los datos de ésta: número secuencial de la compra, fecha, importe total de la compra, cantidad de cajas de cada variedad que se incluye en la compra (al menos una variedad) y si se ha pagado o no con tarjeta.

Semanalmente, los lunes a las 9h, se envía un correo electrónico a los clientes indicando qué variedad está de oferta esa semana. Se desea guardar el histórico de ofertas realizadas. Cada variedad puede tener varias ofertas que se identifican por el año y la semana, tiene una fecha de finalización y un descuento a aplicar a cada caja.

Cada variedad de café se obtiene como resultado de la mezcla de diferentes tipos de grano. Cada tipo de grano se identifica por un código, tiene una descripción y puede ser producido en distintas regiones de algunos países. Cada país se identifica por un código y tiene un nombre. Cada región se identifica por un código, único en el país en que se localiza, y también tiene un nombre. Para conocer la mezcla exacta de cada variedad, hay que almacenar para cada tipo de grano que la compone la cantidad (en gramos de ese grano por cada 100 gramos de la variedad) y el nivel de molido (de 0 a 5).

Adicionalmente se pueden añadir otros aditivos a la variedad. Cada aditivo se identifica por un código y tiene una descripción. Los aditivos pueden ser naturales o artificiales, en el caso de los naturales se quiere almacenar el nombre común y en el caso de los artificiales el código que tiene en la Unión Europea que es único.

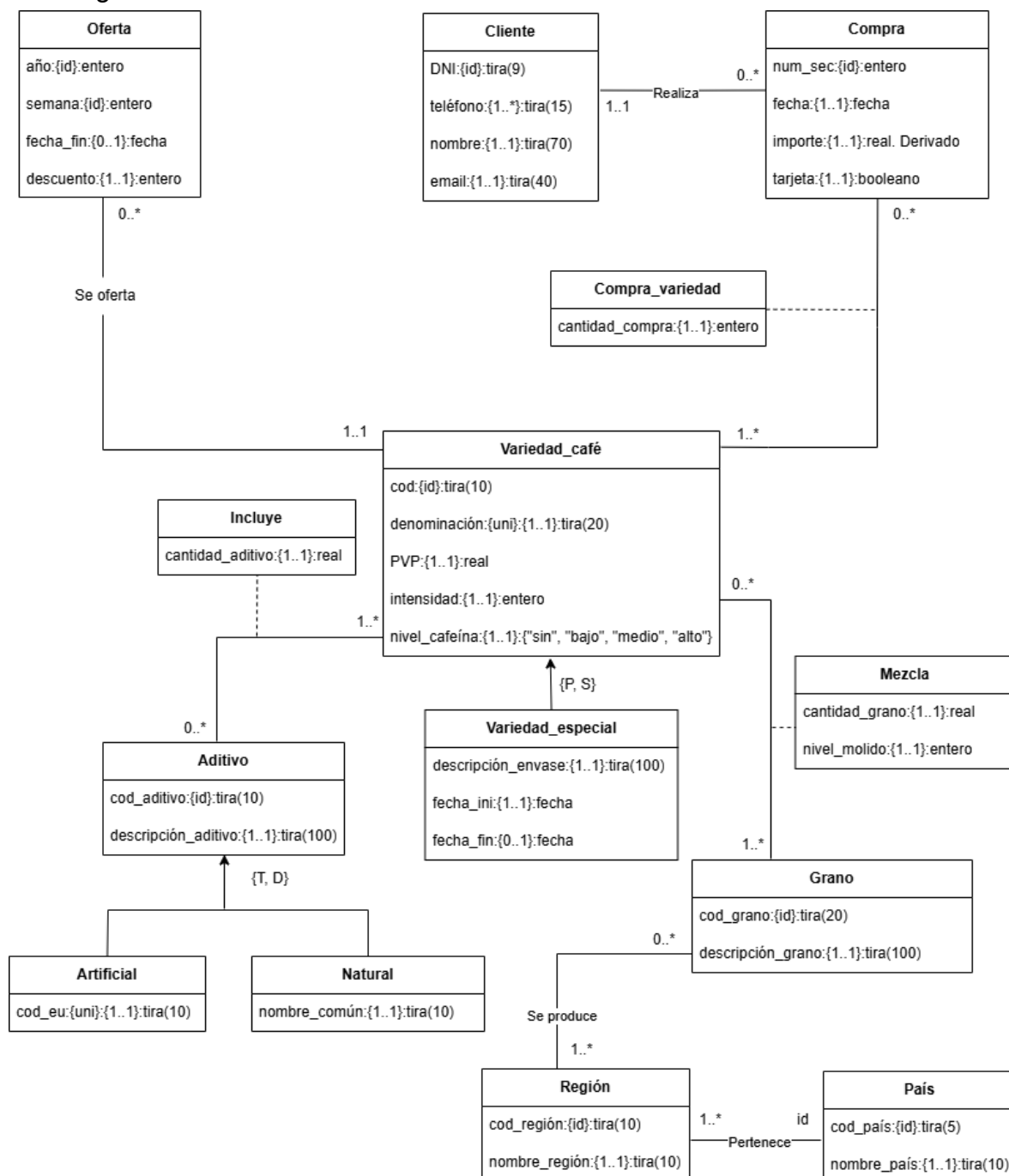
Para cada variedad hay que almacenar qué aditivos incluye y la cantidad (en gramos por cada 100 gramos de variedad).

Periódicamente se elaboran variedades especiales que tienen la misma información que las variedades habituales, pero que llevan distinto envase. En este caso se debe almacenar la descripción del nuevo envase y el período de disponibilidad de dicha variedad, guardando fecha de inicio y fecha de fin.

1. TAREA 1: ANÁLISIS Y DISEÑO CONCEPTUAL

Tarea	Contenido	Puntos	Trabajo a realizar	Contenido de la entrega
1	Análisis del SI Diseño Conceptual	1,5 puntos	- Diseño Conceptual comprobando que el diagrama de clases obtenido satisface todas las reglas del "Test de corrección de un diagrama de clases". - El diagrama de clases debe contener al menos: - una especialización - una clase débil - una clase asociación - un atributo multivaluado - un atributo fecha - un atributo booleano - un atributo derivado	Un solo documento pdf que incluya: - Enunciado del problema. - Diagrama de clases LEGIBLE. - Lista de restricciones de integridad que no se hayan podido expresar en el diagrama (aparte o en el propio diagrama). - Lista de requisitos de proceso y/o consulta.

1.1 Diagrama de clases



1.2 Restricciones de integridad que no se han podido expresar

- 1) El atributo *intensidad* de la clase *Variedad_café* toma un valor entre el 1 y el 12, ambos incluidos.
- 2) El atributo *cantidad_grano* de la clase asociativa *Mezcla* toma un valor real entre el 0 y el 100, es decir, está expresado en cantidad por cada 100 gramos.
- 3) El atributo *nivel_molido* de la clase asociativa *Mezcla* toma un valor entre el 0 y el 5.
- 4) El atributo *cantidad_aditivo* de la clase asociativa *Incluye* toma un valor real entre el 0 y el 100, es decir, está expresado en cantidad por cada 100 gramos.
- 5) En la clase especializada *Variedad_especial* se cumple la condición: *fecha_ini* < *fecha_fin*.
- 6) El atributo *semana* de la clase *Oferta* toma un valor entero entre el 1 y el 52.
- 7) El atributo *importe* de la clase *Compra* toma un valor real con dos dígitos decimales.
- 8) El *importe* de la compra es un valor derivado del sumatorio del producto de las cantidades de las variedades que incluye por su *PVP*.

1.3 Requisitos de proceso y/o consulta

En este apartado se listan algunos requisitos de consulta y/o actualización que se van a considerar.

Requisitos de consulta:

- Consulta de las compras realizadas por un cliente.
- Listar todas las ofertas que ha habido de cada variedad de café, ordenadas por la fecha de finalización.
- Consulta del nivel de molido de los granos que componen una variedad de café.
- Consulta de las regiones en las que se produce un grano específico.

Requisitos de proceso:

- Añadir una compra.
- Calcular el importe total de las compras realizadas por un cliente.
- Eliminar todas las variedades de café que no hayan sido compradas nunca.

2. TAREA 2: DISEÑO LÓGICO

Tarea	Contenido	Puntos	Trabajo a realizar	Contenido de la entrega
2	Diseño Lógico	0,75 puntos	- Diseño lógico. - Normalización.	Un solo documento pdf que incluya: - Resultado de la fase anterior con las correcciones hechas por el profesor. - Esquema relacional textual (con la notación académica vista en clase). - Lista de restricciones de integridad que no se hayan podido representar en el esquema lógico. - Reflexión sobre la Normalización. - Reflexión sobre las directrices de restauración de la integridad referencial. - Explicación de las transacciones para resolver los requisitos de la fase anterior.

2.1 Esquema relacional

Oferta (año:entero, semana:entero, fecha_fin:fecha, descuento:entero, cod:tira(10))

CP:{año, semana}

VNN:{descuento}

Caj:{cod}Referencia Variedad_cafe(cod)

Borrado en cascada

Modificación en cascada

COMP: (semana BETWEEN 1 AND 52)

Cliente (DNI:tira(9), nombre:tira(70), email:tira(40))

CP:{DNI}

VNN:{nombre, email}

Agenda (DNI:tira(9), teléfono:tira(15))

CP:{DNI, teléfono}

Caj:{DNI}Referencia Cliente(DNI)

Borrado en cascada

Modificación en cascada

Compra (num_sec:entero, fecha:fecha, importe:real, tarjeta:booleano, DNI:tira(9))

CP:{num_sec}

VNN:{fecha, importe, tarjeta, DNI}

Caj:{DNI}Referencia Cliente(DNI)

Borrado restrictivo

Modificación en cascada

Variedad_café (cod:tira(10), denominación:tira(20), PVP:real, intensidad:entero, nivel_cafeína:tira(10))

CP:{cod}

Único:{denominación}

VNN:{denominación, PVP, intensidad, nivel_cafeína}

COMP: (intensidad BETWEEN 1 AND 12)

COMP: nivel_cafeína EN('sin', 'bajo', 'medio', 'alto')

Compra_variedad (num_sec:entero, cod:tira(10), cantidad_compra:entero)

CP:{num_sec, cod}

VNN:{cantidad_compra}

CAj:{num_sec} Referencia Compra(num_sec)

Borrado en cascada

Modificación en cascada

CAj:{cod} Referencia Variedad_café(cod)

```

        Borrado restrictivo
        Modificación en cascada
Incluye(cod_aditivo:tira(10),cod:tira(10),cantidad_aditivo:real)
    CP:{cod_aditivo,cod}
    CAj:{cod_aditivo} Referencia Aditivo(cod_aditivo)
        Borrado en casacada
        Modificación en cascada
    CAj:{cod} Referencia Variedad_café(cod)
        Borrado en cascada
        Modificación en cascada
    COMP: (cantidad_aditivo > 0 AND cantidad_aditivo <= 100)
Aditivo(cod_aditivo:tira(10),descripción_aditivo:tira(100))
    CP:{cod_aditivo}
    VNN:{descripción_aditivo}
Artificial(cod_eu:tira(10),cod_aditivo:tira(10))
    CP:{cod_aditivo}
    CAj:{cod_aditivo} Referencia Aditivo(cod_aditivo)
        Borrado en cascada
        Modificación en cascada
    VNN:{cod_eu}
    Único:{cod_eu}
Natural(nombre_común:tira(10),cod_aditivo:tira(10))
    CP:{cod_aditivo}
    Caj:{cod_aditivo} Referencia Aditivo(cod_aditivo)
        Borrado en cascada
        Modificación en cascada
    VNN:{nombre_común}
Grano(cod_grano:tira(20),descripción_grano:tira(100))
    CP:{cod_grano}
    VNN:{descripción_grano}
País(cod_país:tira(5),nombre_país:tira(10))
    CP:{cod_país}
    VNN:{nombre_país}
Región(cod_región:tira(10),cod_país:tira(5),nombre_región:tira(10))
    CP:{cod_país, cod_región}
    VNN:{nombre_región}
    Caj:{cod_país} Referencia País(cod_país)
        Borrado en cascada
        Modificación en cascada
Variedad_especial(descripción_envase:tira(100),fecha_ini:fecha,
fecha_fin:fecha,cod:tira(10))
    CP:{cod}
    Caj:{cod}Referencia Variedad_café(cod)
        Borrado en cascada
        Modificación en cascada
    VNN:{descripción_envase,fecha_ini}
    COMP:(fecha_ini < fecha_fin O fecha_fin es nulo)
Se produce(cod_grano:tira(20),cod_región:tira(10),cod_país:tira(5))

```

```

CP:{cod_grano,cod_país,cod_región}
Caj:{cod_grano}Referencia Grano(cod_grano)
    Borrado en cascada
    Modificación en cascada
Caj:{cod_país,cod_región} Referencia Región(cod_país,
cod_región)
    Borrado en cascada
    Modificación en cascada
Mezcla(cod:tira(10),cod_grano:tira(20),cantidad_grano:real,
nivel_molido:entero)
    CP:{cod,cod_grano}
    VNN:{nivel_molido,cantidad_grano}
    CAj:{cod}Referencia Variedad_café(cod)
        Borrado en cascada
        Modificación en cascada
    Caj:{cod_grano}Referencia Grano(cod_grano)
        Borrado restrictivo
        Modificación en cascada
COMP: (cantidad_grano > 0 AND cantidad_grano <= 100)
COMP: (nivel_molido BETWEEN 0 AND 5)

```

2.2 Restricciones de integridad que no se han podido expresar

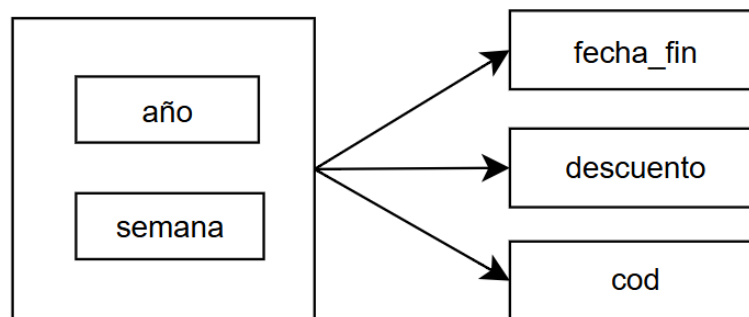
- 1) Todo cliente debe aparecer al menos una vez en *Agenda*.
- 2) Todo valor que aparezca en el atributo *cod_aditivo* de *Aditivo* debe aparecer en el atributo *cod_Aditivo* de *Artificial* o *Natural*.
- 3) No puede haber un mismo valor en el atributo *cod_aditivo* de *Artificial* y en el *cod_aditivo* de *Natural*.
- 4) El cálculo del importe de una compra se realiza según lo explicado en la descripción del sistema de información.
- 5) El atributo *importe* de la clase *Compra* toma un valor real con dos dígitos decimales.
- 6) Todo valor de *cod_grano* de la relación *Grano* aparece en al menos una vez en el atributo *cod_grano* de la relación *Se_produce*.
- 7) Todo valor de *num_sec* de la relación *Compra* aparece al menos una vez en el atributo *num_sec* de la relación *Compra_variedad*.
- 8) Todo valor de *cod* de la relación *Variedad_café* aparece al menos una vez en el atributo *cod* de la relación *Mezcla*.
- 9) Para todo valor de *cod_país* de la relación *País* debe existir al menos un valor de *cod_país* en la relación *Región*.
- 10) Todo valor de *cod_aditivo* de la relación *Aditivo* aparece al menos una vez en el atributo *cod_aditivo* de la relación *Incluye*.

2.3 Normalización

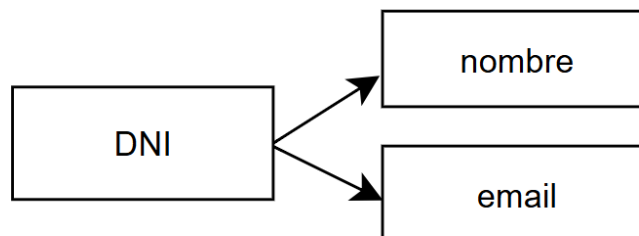
- Una relación R está en 1FN si sus atributos sólo pueden tomar valores atómicos (simples, indivisibles).
- Una relación R está en 2FN si está en 1FN y todo atributo no-primo depende funcionalmente de forma completa de todas las claves de R.
- Una relación R está en 3FN si está en 2FN y todas las dependencias funcionales $X \rightarrow A$ del conjunto de interés cumplen que:
 - X es una clave, o
 - A es un atributo primo

Para cada relación se va a dibujar el diagrama de dependencias funcionales para estudiar si están en 3FN.

- Relación *Oferta*: está en 3FN



- Está en 1FN: no hay tipos estructurados.
 - Está en 2FN: todo atributo no-primo (*fecha_fin*, *descuento* y *cod*) dependen de forma completa de la única clave de la relación {*año*, *semana*}.
 - Está en 3FN: en las tres dependencias funcionales se cumple {*año*, *semana*} es una clave.
- Relación *Cliente*: está en 3FN

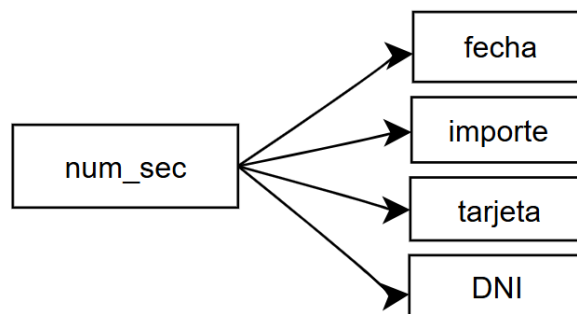


- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*nombre* y *email*) dependen de forma completa de la única clave de la relación {*DNI*}.
- Está en 3FN: en las dos dependencias funcionales se cumple {*DNI*} es una clave.

- Relación *Agenda*: está en 3FNG

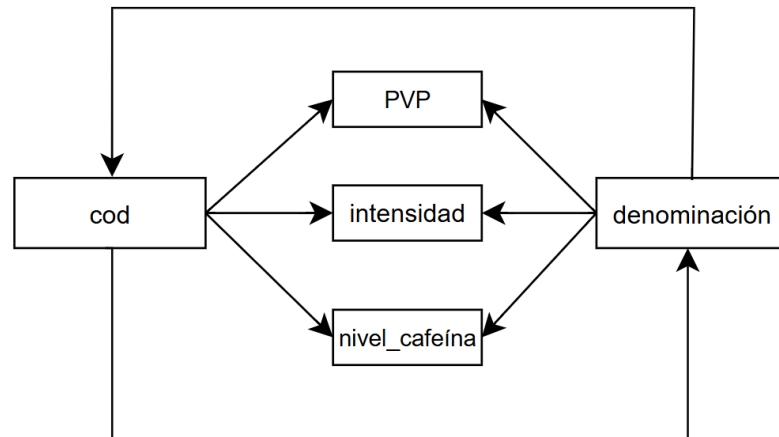


- Está en 1FN: No hay tipos estructurados.
 - Está en 2FNG y 3FNG: no hay dependencias funcionales.
- Relación *Compra*: está en 3FN

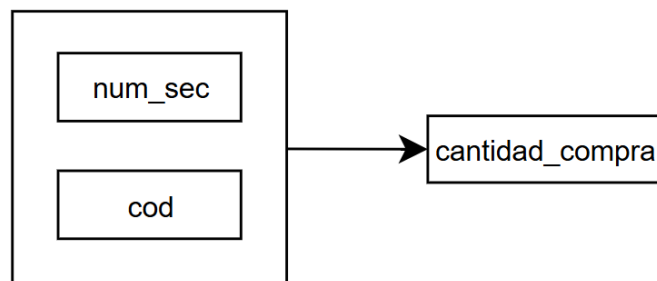


- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*fecha*, *importe*, *tarjeta* y *DNI*) dependen de forma completa de la única clave de la relación {*num_sec*}.
- Está en 3FN: en las cuatro dependencias funcionales se cumple {*num_sec*} es una clave.

- Relación *Variedad_café*: está en 3FN

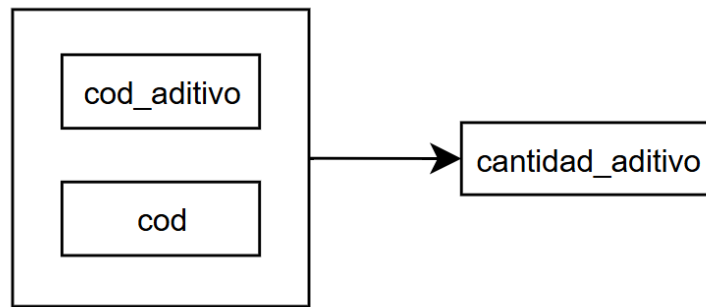


- Está en 1FN: no hay tipos estructurados.
 - Está en 2FN: todo atributo no-primo (*denominación*, *PVP*, *intensidad* y *nivel_cafeína*) dependen de forma completa de las dos claves de la relación {*cod*} y {*denominación*}.
 - Está en 3FN: en las todas las dependencias funcionales se cumple la primera de las condiciones de la 3FN, en las dependencias entre *cod* y *denominación* se cumplen las dos condiciones.
- Relación *Compra_variedad*: está en 3FN



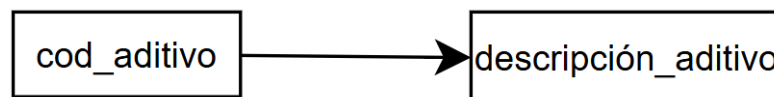
- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*cantidad_compra*) dependen de forma completa de la única clave de la relación {*num_sec*, *cod*}.
- Está en 3FN: en la dependencia funcional se cumple {*num_sec*, *cod*} es una clave.

- Relación *Incluye*: está en 3FN



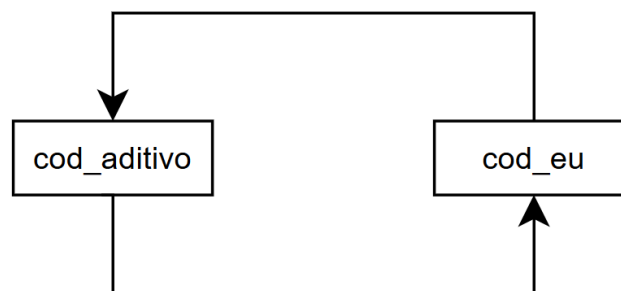
- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primario (*cantidad_aditivo*) dependen de forma completa de la única clave de la relación {*cod_aditivo*, *cod*}.
- Está en 3FN: en la dependencia funcional se cumple {*cod_aditivo*, *cod*} es una clave.

- Relación *Aditivo*: está en 3FN



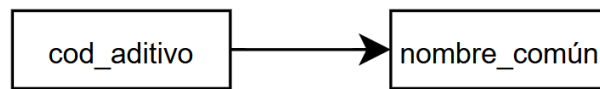
- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primario (*descripción_aditivo*) dependen de forma completa de la única clave de la relación {*cod_aditivo*}.
- Está en 3FN: en la dependencia funcional se cumple {*cod_aditivo*} es una clave.

- Relación *Artificial*: está en 3FNG

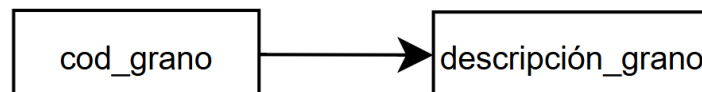


- Está en 1FN: no hay tipos estructurados.
- Está en 2FNG: no hay atributos no-primarios.
- Está en 3FNG: en las dependencias entre *cod_aditivo* y *cod_eu* se cumplen las dos condiciones.

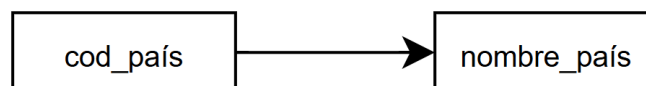
- Relación *Natural*: está en 3FN



- Está en 1FN: no hay tipos estructurados.
 - Está en 2FN: todo atributo no-primo (*nombre_común*) depende de forma completa de la única clave de la relación {*cod_aditivo*}.
 - Está en 3FN: en la dependencia funcional se cumple {*cod_aditivo*} es una clave.
- Relación *Grano*: está en 3FN

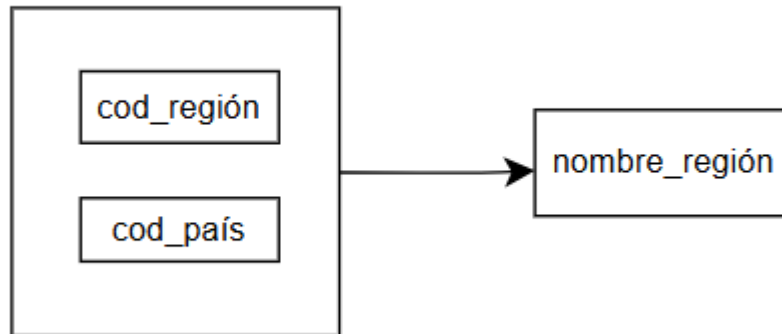


- Está en 1FN: no hay tipos estructurados.
 - Está en 2FN: todo atributo no-primo (*descripción_grano*) depende de forma completa de la única clave de la relación {*cod_grano*}.
 - Está en 3FN: en la dependencia funcional se cumple {*cod_grano*} es una clave.
- Relación *País*: está en 3FN

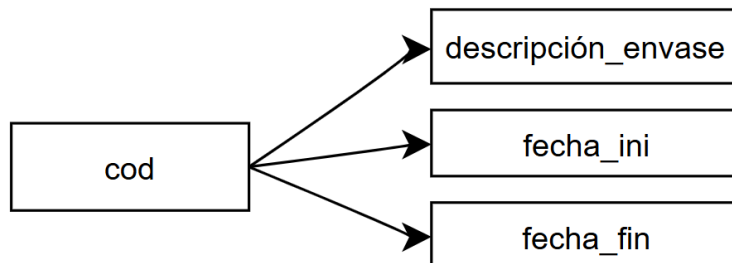


- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*nombre_país*) depende de forma completa de la única clave de la relación {*cod_país*}.
- Está en 3FN: en la dependencia funcional se cumple {*cod_país*} es una clave.

- Relación *Región*: está en 3FN

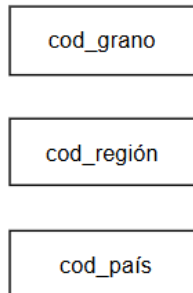


- Está en 1FN: no hay tipos estructurados.
 - Está en 2FN: todo atributo no-primo (*nombre_región*) depende de forma completa de la única clave de la relación {*cod_país*, *cod_región*}.
 - Está en 3FN: en la dependencia funcional se cumple {*cod_país*, *cod_región*} es una clave.
- Relación *Variedad_especial*: está en 3FN

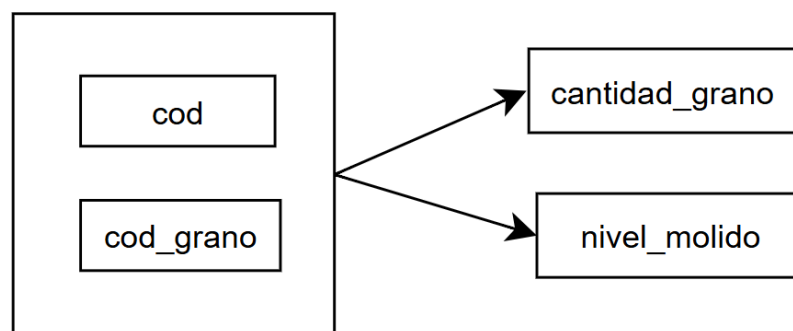


- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*descripción_envase*, *fecha_ini* y *fecha_fin*) dependen de forma completa de la única clave de la relación {*cod*}.
- Está en 3FN: en las tres dependencias funcionales se cumple {*cod*} es una clave.

- Relación *Se_produce*: está en 3FNG



- Está en 1FN: no hay tipos estructurados.
 - Está en 2FNG y 3FNG: no hay dependencias funcionales.
- Relación *Mezcla*: está en 3FN



- Está en 1FN: no hay tipos estructurados.
- Está en 2FN: todo atributo no-primo (*cantidad_grano* y *nivel_molido*) dependen de forma completa de la única clave de la relación {*cod*, *cod_grano*}.
- Está en 3FN: en las tres dependencias funcionales se cumple {*cod*, *cod_grano*} es una clave.

2.4 Directrices de restauración de la integridad referencial

Aunque ya se han definido las directrices en el esquema relacional, en este apartado se repasan una a una explicando las implicaciones que tienen las directrices elegidas.

- Relación *Oferta*:
 - Clave ajena a *Variedad_café*: Borrado en cascada y modificación en cascada. Si se borra una variedad de café, también se borrarán las ofertas asociadas a dicha variedad. Si se modifica el código de una variedad de café, el cambio se trasladará a las ofertas correspondientes.
- Relación *Agenda*:
 - Clave ajena a *Cliente*: Borrado en cascada y modificación en cascada. Si se borra un cliente, también se eliminarán los teléfonos asociados a dicho cliente en la agenda. Si se modifica el DNI de un cliente, se trasladará el cambio a los registros en la agenda.
- Relación *Compra*:
 - Clave ajena a *Cliente*: Borrado restrictivo y modificación en cascada. No se puede borrar un cliente si existen compras asociadas al cliente. Si se modifica el DNI de un cliente, se trasladará el cambio a las compras realizadas por dicho cliente.
- Relación *Compra_variedad*:
 - Clave ajena a *Compra*: Borrado en cascada y modificación en cascada. Si se borra una compra también se borrarán las variedades asociadas a dicha compra. Si se modifica el número de la compra, se trasladará el cambio en los registros de *compra_variedad*
 - Clave ajena a *Variedad_café*: Borrado restrictivo y modificación en cascada. No se puede borrar una variedad de café si está asociada a alguna compra mediante la relación *Compra_variedad*. Si se modifica el código de la variedad, se trasladará el cambio a *compra_variedad*.
- Relación *Incluye*:
 - Clave ajena a *Aditivo*: Borrado en cascada y modificación en cascada. Si se borra un aditivo, también se borrarán las relaciones que lo incluyan en variedad de café. Si se modifica el código del aditivo, se trasladará el cambio a la relación *Incluye*.
 - Clave ajena a *Variedad_café*: Borrado en cascada y modificación en cascada. Si se borra una variedad de café, también se borrarán los aditivos asociados a dicha variedad. Si se modifica el código de la variedad de café, se trasladará el cambio a la relación *Incluye*.

- Relación *Artificial*:
 - Clave ajena a *Aditivo*: Borrado en cascada y modificación en cascada. Si se borra un aditivo, también se borrarán en *Artificial*. Si se modifica el código de aditivo, se trasladará el cambio a *Artificial*.
- Relación *Natural*:
 - Clave ajena a *Aditivo*: Borrado en cascada y modificación en cascada. Si se borra un aditivo, también se borrarán en *Natural*. Si se modifica el código de aditivo, se trasladará el cambio a *Natural*.
- Relación *Región*:
 - Clave ajena a *País*: Borrado en cascada y modificación en cascada. Si se borra un país, también se borrarán las regiones asociadas a ese país en *Región*. Si se modifica el código de un país, se trasladará el cambio a *Región*.
- Relación *Variedad_especial*:
 - Clave ajena a *Variedad_cafe*: Borrado en cascada y modificación en cascada. Si se borra una variedad de café, también se borrarán su información asociada en *Variedad_especial*. Si se modifica el código de la variedad de café, se trasladará el cambio a *Variedad_especial*.
- Relación *Se_produce*:
 - Clave ajena a *Grano*: Borrado en cascada y modificación en cascada. Si se borra un grano, también se borrarán las relaciones que indican dónde se produce dicho grano. Si se modifica el código del grano, se trasladará el cambio a la relación *Se_produce*.
 - Clave ajena a *Región*: Borrado en cascada y modificación en cascada. Si se borra una región, también se borrarán los tipos de granos producidos en dicha región. Si se modifica el código de la región, se trasladará el cambio a la relación *Se_produce*.
- Relación *Mezcla*:
 - Clave ajena a *Variedad_café*: Borrado en cascada y modificación en cascada. Si se borra una variedad de café, también se borrarán las mezclas asociadas a esa variedad de café. Si se modifica el código de la variedad, se trasladará el cambio a la relación *Mezcla*.
 - Clave ajena a *Grano*: Borrado restrictivo y modificación en cascada. No se puede borrar un grano si está asociado a alguna variedad de café mediante la relación *Mezcla*. Si se modifica el código del grano, se trasladará el cambio a la relación *Mezcla*.

2.5 Transacciones

- Transacción 1 (T1): consulta de las compras realizadas por un cliente: será necesario buscar el cliente por su DNI y mostrar sus compras.
- Transacción 2 (T2): listar todas las ofertas que ha habido de cada variedad de café, ordenadas por la fecha de finalización.
- Transacción 3 (T3): consulta del nivel de molido de los granos que componen una variedad de café: se buscará la variedad de café por su código y se listará, para cada grano de su mezcla, el nivel de molido correspondiente.
- Transacción 4 (T4): consulta de las regiones en las que se produce un grano específico: se buscará el grano por su código y se listarán las regiones donde se produce.
- Transacción 5 (T5): añadir una compra: se buscará el cliente asociado a la compra y se buscará el PVP de cada variedad de café comprada, sumando los productos de la cantidad comprada por el precio de cada variedad para obtener el importe de la compra.
- Transacción 6 (T6): calcular el importe total de las compras realizadas por un cliente: se busca a un cliente por su DNI y se realiza el sumatorio de todas las compras asociadas a ese cliente.
- Transacción 7 (T7): eliminar todas las variedades de café que no hayan sido compradas nunca: se buscan aquellas variedades de café cuyo código no aparezca en la relación *Compra_variedad* y se procederá a su eliminación.

3. TAREA 3: CREACIÓN Y CARGA DE LA BASE DE DATOS EN ORACLE

Tarea	Contenido	Puntos	Trabajo a realizar	Contenido de la entrega
3	Creación de la base de datos en Oracle	1 punto	- Creación de la base de datos en Oracle con el usuario Oracle del grupo. - Carga de la base datos.	Un solo documento pdf que incluya: - Resultado de las fases anteriores con las correcciones hechas por el profesor. - Instrucciones de creación de la base de datos. - Diagrama Oracle de la base de datos generado con el <i>Data Modeler</i>. - Explicación de cómo se ha realizado la carga de datos

3.1 Instrucciones de la creación de la base de datos

En relación a las directrices de restauración de la integridad referencial ante la modificación de una clave apuntada por una clave ajena, en Oracle sólo existe la directriz MODIFICACIÓN RESTRICTIVA, para resolver este problema, las claves ajenas se definirán en todos los casos como diferibles inicialmente diferidas, con esto se podrán definir procedimientos que actualicen las claves ajenas en cascada.

3.1.1 Creación de la tabla CLIENTE

```
CREATE TABLE Cliente (
    DNI          VARCHAR2(9),
    nombre       VARCHAR2(70) NOT NULL,
    email        VARCHAR2(40) NOT NULL,
    CONSTRAINT PK_Cliente PRIMARY KEY (DNI)
);
```

3.1.2 Creación de la tabla AGENDA

```
CREATE TABLE Agenda (
    DNI          VARCHAR2(9),
    telefono     VARCHAR2(15),
    CONSTRAINT PK_Agenda PRIMARY KEY (DNI, telefono),
    CONSTRAINT FK_Agenda_Cliente FOREIGN KEY (DNI)
        REFERENCES Cliente(DNI)
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED
);
```

3.1.3 Creación de la tabla VARIEDAD_CAFÉ

```
CREATE TABLE Variedad_cafe (
    cod          VARCHAR2(10),
    denominacion VARCHAR2(20) NOT NULL,
    PVP          NUMBER NOT NULL,
    intensidad   NUMBER NOT NULL,
    nivel_cafeina VARCHAR2(10) NOT NULL,
    CONSTRAINT PK_Variedad PRIMARY KEY (cod),
    CONSTRAINT UQ_Denominacion UNIQUE (denominacion),
    CONSTRAINT CK_Intensidad CHECK (intensidad BETWEEN 1 AND 12),
);
```

```
CONSTRAINT CK_NivelCafeina CHECK (  
    nivel_cafeina IN ('sin','bajo','medio','alto')  
)  
);
```

3.1.4 Creación de la tabla OFERTA

```
CREATE TABLE Oferta (  
    anio          NUMBER,  
    semana        NUMBER,  
    fecha_fin     DATE,  
    descuento     NUMBER NOT NULL,  
    cod           VARCHAR2(10),  
    CONSTRAINT PK_Oferta PRIMARY KEY (anio, semana),  
    CONSTRAINT CK_Semana CHECK (semana BETWEEN 1 AND 52),  
    CONSTRAINT FK_Oferta_Variedad FOREIGN KEY (cod)  
        REFERENCES Variedad_cafe(cod)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.5 Creación de la tabla COMPRA

```
CREATE TABLE Compra (  
    num_sec       NUMBER,  
    fecha         DATE NOT NULL,  
    importe       NUMBER NOT NULL,  
    tarjeta       CHAR(1) NOT NULL,  
    DNI           VARCHAR2(9) NOT NULL,  
    CONSTRAINT PK_Compra PRIMARY KEY (num_sec),  
    CONSTRAINT CK_Tarjeta CHECK (tarjeta IN ('S','N')),  
    CONSTRAINT FK_Compra_Cliente FOREIGN KEY (DNI)  
        REFERENCES Cliente(DNI)  
        DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.6 Creación de la tabla COMPRA_VARIEDAD

```
CREATE TABLE Compra_variedad (  
    num_sec       NUMBER,  
    cod           VARCHAR2(10),  
    cantidad_compra NUMBER NOT NULL,  
    CONSTRAINT PK_CompraVariedad PRIMARY KEY (num_sec, cod),  
    CONSTRAINT FK_CV_Compra FOREIGN KEY (num_sec)  
        REFERENCES Compra(num_sec)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT FK_CV_Variedad FOREIGN KEY (cod)  
        REFERENCES Variedad_cafe(cod)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.7 Creación de la tabla ADITIVO

```
CREATE TABLE Aditivo (  
    cod_aditivo          VARCHAR2(10),  
    descripcion_aditivo  VARCHAR2(100) NOT NULL,  
    CONSTRAINT PK_Aditivo PRIMARY KEY (cod_aditivo)  
);
```

3.1.8 Creación de la tabla ARTIFICIAL

```
CREATE TABLE Artificial (  
    cod_aditivo VARCHAR2(10),  
    cod_eu       VARCHAR2(10) NOT NULL,  
    CONSTRAINT PK_Artificial PRIMARY KEY (cod_aditivo),  
    CONSTRAINT UQ_CodeEU UNIQUE (cod_eu),  
    CONSTRAINT FK_Artificial_Aditivo FOREIGN KEY (cod_aditivo)  
        REFERENCES Aditivo(cod_aditivo)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.9 Creación de la tabla NATURAL

```
CREATE TABLE Natural (  
    cod_aditivo  VARCHAR2(10),  
    nombre_comun VARCHAR2(10) NOT NULL,  
    CONSTRAINT PK_Natural PRIMARY KEY (cod_aditivo),  
    CONSTRAINT FK_Natural_Aditivo FOREIGN KEY (cod_aditivo)  
        REFERENCES Aditivo(cod_aditivo)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.10 Creación de la tabla INCLUYE

```
CREATE TABLE Incluye (  
    cod_aditivo  VARCHAR2(10),  
    cod          VARCHAR2(10),  
    cantidad_aditivo NUMBER,  
    CONSTRAINT PK_Incluye PRIMARY KEY (cod_aditivo, cod),  
    CONSTRAINT CK_CantidadAditivo CHECK (  
        cantidad_aditivo > 0 AND cantidad_aditivo <= 100  
    ),  
    CONSTRAINT FK_Incluye_Aditivo FOREIGN KEY (cod_aditivo)  
        REFERENCES Aditivo(cod_aditivo)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT FK_Incluye_Variedad FOREIGN KEY (cod)  
        REFERENCES Variedad_cafe(cod)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.11 Creación de la tabla GRANO

```
CREATE TABLE Grano (  
    cod_grano          VARCHAR2(20),  
    descripcion_grano  VARCHAR2(100) NOT NULL,  
    CONSTRAINT PK_Grano PRIMARY KEY (cod_grano)  
);
```

3.1.12 Creación de la tabla PAÍS

```
CREATE TABLE Pais (  
    cod_pais          VARCHAR2(5),  
    nombre_pais       VARCHAR2(10) NOT NULL,  
    CONSTRAINT PK_Pais PRIMARY KEY (cod_pais)  
);
```

3.1.13 Creación de la tabla REGIÓN

```
CREATE TABLE Region (  
    cod_pais          VARCHAR2(5),  
    cod_region        VARCHAR2(10),  
    nombre_region     VARCHAR2(10) NOT NULL,  
    CONSTRAINT PK_Region PRIMARY KEY (cod_pais, cod_region),  
    CONSTRAINT FK_Region_Pais FOREIGN KEY (cod_pais)  
        REFERENCES Pais(cod_pais)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.14 Creación de la tabla SE_PRODUCIR

```
CREATE TABLE Se_produce (  
    cod_grano          VARCHAR2(20),  
    cod_pais           VARCHAR2(5),  
    cod_region         VARCHAR2(10),  
    CONSTRAINT PK_SeProduce PRIMARY KEY (cod_grano, cod_pais,  
cod_region),  
    CONSTRAINT FK_SP_Grano FOREIGN KEY (cod_grano)  
        REFERENCES Grano(cod_grano)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT FK_SP_Region FOREIGN KEY (cod_pais, cod_region)  
        REFERENCES Region(cod_pais, cod_region)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED  
);
```

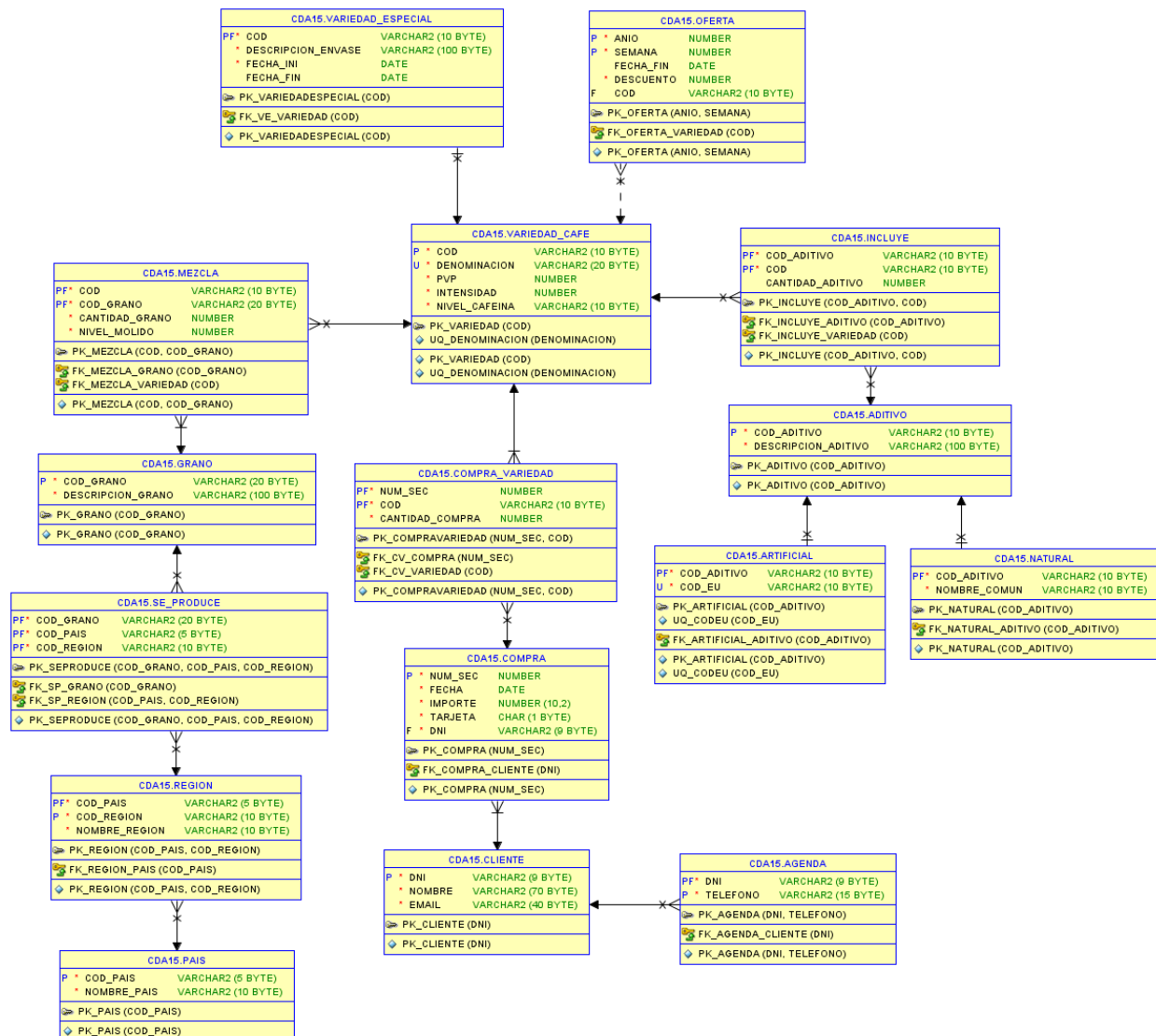
3.1.15 Creación de la tabla MEZCLA

```
CREATE TABLE Mezcla (  
    cod                VARCHAR2(10),  
    cod_grano          VARCHAR2(20),  
    cantidad_grano     NUMBER NOT NULL,  
    nivel_molido       NUMBER NOT NULL,  
    CONSTRAINT PK_Mezcla PRIMARY KEY (cod, cod_grano),  
    CONSTRAINT CK_CantidadGrano CHECK (  
        cantidad_grano > 0 AND cantidad_grano <= 100  
    ),  
    CONSTRAINT CK_Molido CHECK (nivel_molido BETWEEN 0 AND 5),  
    CONSTRAINT FK_Mezcla_Variedad FOREIGN KEY (cod)  
        REFERENCES Variedad_cafe(cod)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT FK_Mezcla_Grano FOREIGN KEY (cod_grano)  
        REFERENCES Grano(cod_grano)  
        DEFERRABLE INITIALLY DEFERRED  
);
```

3.1.16 Creación de la tabla VARIEDAD_ESPECIAL

```
CREATE TABLE Variedad_especial (  
    cod                VARCHAR2(10),  
    descripcion_envase VARCHAR2(100) NOT NULL,  
    fecha_ini          DATE NOT NULL,  
    fecha_fin          DATE,  
    CONSTRAINT PK_VariedadEspecial PRIMARY KEY (cod),  
    CONSTRAINT FK_VE_Variedad FOREIGN KEY (cod)  
        REFERENCES Variedad_cafe(cod)  
        ON DELETE CASCADE DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT CK_Fechas CHECK (  
        fecha_fin IS NULL OR fecha_ini < fecha_fin  
    )  
);
```

3.2 Diagrama generado por Oracle con el Data Modeler



3.3 Carga de la base de datos

El lenguaje PL/SQL de Oracle dispone del paquete de generación aleatoria de valores DBMS_RANDOM que tiene varias funciones que devuelven valores aleatorios. A continuación se presentan algunas de estas funciones para obtener valores de diferente tipo:

- Valor entero de 5 cifras, por ejemplo (en Oracle NUMBER(5, 0)):
 - `trunc(dbms_random.value(1, 99999))`: devuelve un valor tal que $1 \leq \text{valor} < 99999$
- Valor real hasta un millón y con dos decimales, por ejemplo (en Oracle NUMBER(8, 2)):
 - `round(dbms_random.value(1, 999999), 2)`: devuelve un valor con dos decimales tal que $1 \leq \text{valor} < 999999$
- Valor alfanumérico de 100 caracteres (en ORACLE VARCHAR2(100)):
 - `dbms_random.string('a', 100)`

- Valor de tipo fecha entre dos fechas dadas como tiras de caracteres '01/01/2001' y '31/12/2022': la expresión siguiente utiliza la función to_date para convertir una tira de caracteres en un valor de tipo fecha, resta las dos fechas obteniendo la cantidad de días que hay entre ellas, obtiene un entero entre 0 y esa cantidad y suma esa cantidad de días a la fecha menor:
 - to_date('01/01/2001') + trunc(dbms_random.value(0, to_date('31/12/2022') – to_date('01/01/2001')))

Obviamente la carga usando funciones aleatorias genera datos no muy elegantes pero esto no es relevante para alcanzar los objetivos deseados, a pesar de esto, en algunos casos se ha realizado una carga un poco más elaborada para conseguir datos más reales.

3.3.1 Funciones de carga

Función CREAR_COD_PAIS

Genera un código alfanumérico único para identificar cada país a partir de la secuencia.

```
create or replace function crear_cod_pais
return varchar2
is
begin
    return 'p_' || lpad(seq_cod_pais.nextval, 3, '0');
end;
```

Función CREAR_COD_EU

Genera un código alfanumérico único para identificar cada entidad europea.

```
create or replace function crear_cod_eu
return varchar2
is
begin
    return 'eu_' || lpad(seq_cod_eu.nextval, 5, '0');
end;
```

Función CREAR_NUM_COMPRA

Genera un número único para identificar cada compra.

```
create or replace function crear_num_compra
return number
is
begin
    return seq_compra_num.nextval;
end;
```

Función CREAR_COD_VARIEDAD

Genera un código alfanumérico único para identificar cada variedad.

```
create or replace function crear_cod_variedad
return varchar2
is
begin
    return 'var_' || lpad(seq_variedad_cod.nextval, 5, '0');
end;
```

Función CREAR_COD_ADITIVO

Genera un código alfanumérico único para identificar cada aditivo.

```
create or replace function crear_cod_aditivo
return varchar2
is
begin
    return 'adit_' || lpad(seq_aditivo_cod.nextval, 3, '0');
end;
```

Función CREAR_COD_REGION

Genera un código alfanumérico único para identificar cada región.

```
create or replace function crear_cod_region
return varchar2
is
begin
    return 'reg_' || lpad(seq_region_cod.nextval, 3, '0');
end;
```

Función CREAR_COD_GRANO

Genera un código alfanumérico único para identificar cada tipo de grano.

```
create or replace function crear_cod_grano
return varchar2
is
begin
    return 'gra_' || lpad(seq_grano_cod.nextval, 5, '0');
end;
```

Función CREAR_DESCRIPCION_ENVASE

Genera una cadena aleatoria de 100 caracteres para describir el envase del producto.

```
create or replace function crear_descripcion_envase
return varchar2
is
begin
    return dbms_random.string('a', 100);
end;
```

Función CREAR_DESCRIPCION_GRANO

Genera una descripción aleatoria de 100 caracteres sobre las características del grano.

```
create or replace function crear_descripcion_grano
return varchar2
is
begin
    return dbms_random.string('a', 100);
end;
```

Función CREAR_DESCRIPCION_ADITIVO

Crea una descripción aleatoria de 100 caracteres para cada aditivo.

```
create or replace function crear_descripcion_aditivo
return varchar2
is
begin
    return dbms_random.string('a', 100);
end;
```

Función CREAR_CANTIDAD_GRANO

Devuelve un valor numérico real entre 1 y 100. Se utiliza para representar la cantidad de un grano específico por cada 100 gramos de la variedad de café.

```
create or replace function crear_cantidad_grano
return number
is
begin
    return round(dbms_random.value(1, 100), 2);
end;
```

Función CREAR_CANTIDAD_ADITIVO

Genera un número real entre 1 y 100, indicando la proporción de aditivos por cada 100 gramos de producto.

```
create or replace function crear_cantidad_aditivo
return number
is
begin
    return round(dbms_random.value(1, 100), 2);
end;
```

Función CREAR_CANTIDAD_COMPRA

Devuelve un número entero entre 1 y 10 para determinar cuántas cajas de una variedad específica se incluyen en una transacción.

```
create or replace function crear_cantidad_compra
return number
is
begin
    return trunc(dbms_random.value(1, 11));
end;
```

Función CREAR_NIVEL_CAFEINA

Esta función genera el nivel de cafeína de forma aleatoria entre las cuatro opciones disponibles (sin, bajo, medio y alto).

```
create or replace function crear_nivel_cafeina
return varchar2
is
    v_random number;
    v_nivel varchar2(10);
begin
    v_random := trunc(dbms_random.value(1, 5));
    IF v_random = 1 then v_nivel := 'sin';
        elsif v_random = 2 then v_nivel := 'bajo';
        elsif v_random = 3 then v_nivel := 'medio';
        else v_nivel := 'alto';
    end if;
    return v_nivel;
end;
```

Función TARJETA

Esta función devuelve 1 o 0 dependiendo si ha utilizado o no tarjeta respectivamente.

```
create or replace function tarjeta
return char
is
begin
    if trunc(dbms_random.value(0, 2)) = 0 then
        return 'N';
    else
        return 'S';
    end if;
end;
```

Función CREAR_DENOMINACION

Genera una denominación única para cada variedad de café utilizando una secuencia de base de datos.

```
create or replace function crear_denominacion
return varchar2
is
begin
    return 'variedad_' || seq_denominacion.nextval;
end;
```

Función CREAR_NOMBRE

Esta función genera un nombre para las regiones, países y aditivos naturales.

```
create or replace function crear_nombre
return varchar2
is
begin
```

```
    return dbms_random.string('a', 10);  
end;
```

Función CREAR_FECHA

Esta función genera una fecha para una compra y una fecha de fin para oferta.

```
create or replace function crear_fecha  
return date  
is  
begin  
    return to_date('01/01/2001') +  
        trunc(dbms_random.value(0, to_date('31/12/2025') -  
to_date('01/01/2001')));  
end;
```

Función CREAR_FECHA_FIN

Esta función genera una fecha de fin de variedad_especial, obteniendo como parámetro su fecha de inicio y devolviendo una fecha mayor o igual a la fecha de inicio.

```
create or replace function crear_fecha_fin(p_fecha_ini DATE)  
return date  
is  
begin  
    return p_fecha_ini + trunc(dbms_random.value(0, 180));  
end;
```

Función CREAR_ANIO

Genera un número entero aleatorio entre 2001 y 2025. Se emplea principalmente como parte de la clave primaria de la tabla Oferta para identificar el periodo anual de la promoción.

```
create or replace function crear_anio  
return number  
is  
begin  
    return trunc(dbms_random.value(2001, 2026));  
end;
```

Función CREAR_DESCUENTO

Genera un valor entero entre 1 y 100. Representa el porcentaje o valor de rebaja a aplicar a cada caja de café durante una oferta semanal.

```
create or replace function crear_descuento  
return number  
is  
begin  
    return trunc(dbms_random.value(1, 101));  
end;
```

Función SEMANA

Genera un número entero entre 1 y 52. Identifica la semana del año en la que se lanza una oferta.

```
create or replace function semana
return number
is
begin
    return trunc(dbms_random.value(1, 53));
end;
```

Función CREAR_PVP

Genera un valor numérico real con dos decimales para el atributo PVP de Variedad_café. Representa el precio de venta al público de una caja de 10 unidades.

```
create or replace function crear_pvp
return number
is
begin
    return round(dbms_random.value(1, 20), 2);
end;
```

Función CREAR_INTENSIDAD

Devuelve un número entero entre 1 y 12 como indica la restricción de integridad. Define la fuerza del sabor de una variedad de café.

```
create or replace function crear_intensidad
return number
is
begin
    return trunc(dbms_random.value(1, 13));
end;
```

Función CREAR_NIVEL_MOLIDO

Genera un valor entero entre 0 y 5. Indica el grado de molienda del grano dentro de una mezcla específica.

```
create or replace function crear_nivel_molido
return number
is
begin
    return trunc(dbms_random.value(0, 6));
end;
```

Función CALCULAR_IMPORTE

Función que suma el coste total de una compra específica. Multiplica la cantidad de cajas compradas por el PVP de cada variedad asociada.

```
create or replace function calcular_importe(p_num_sec number)
return number
is
    v_importe number;
begin
    select sum(cv.cantidad_compra * v.pvp)
    into v_importe
    from compra_variedad cv
    join variedad_cafe v on cv.cod = v.cod
    where cv.num_sec = p_num_sec;

    return round(v_importe, 2);
end;
```

3.3.2 Secuencias

Una secuencia es un objeto de base de datos que permite la generación de números secuenciales y son útiles entre otras cosas para generar identificadores únicos.

En este proyecto se han definido las siguientes secuencias:

- Secuencia para generar números consecutivos para el identificador de las compras.
`CREATE SEQUENCE SEQ_NUM_COMPRA INCREMENT BY 1 START WITH 1;`
- Secuencia para generar números consecutivos para el código de las variedades de café.
`CREATE SEQUENCE seq_variedad_cod START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para la denominación de las variedades de café.
`CREATE SEQUENCE seq_denominacion START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para el código de los países.
`CREATE SEQUENCE seq_pais_cod START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para el código de los granos de café.
`CREATE SEQUENCE seq_grano_cod START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para el código de los aditivos.
`CREATE SEQUENCE seq_aditivo_cod START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para el código de los aditivos artificiales.
`CREATE SEQUENCE seq_artificial_cod START WITH 1 INCREMENT BY 1;`
- Secuencia para generar números consecutivos para el número de compra.
`CREATE SEQUENCE seq_compra_num START WITH 1 INCREMENT BY 1;`

- Secuencia para generar números consecutivos para el código de las regiones.

```
CREATE SEQUENCE seq_region_cod START WITH 1 INCREMENT BY 1;
```

3.3.3 Carga de la tabla CLIENTE

La carga de esta tabla es sencilla, características:

- Número de filas: 10000: el bucle va hasta 10000.
- Columnas:
 - *dni*: generado con la función *fun.crear_dni*.
 - *nombre*: generado con la función *fun.crear.nombre*.
 - *email*: generado con la función *fun.crear_email*.

```
declare
    v_dni cliente.dni%type;
    v_existe_dni number;
begin
    for i in 1..10000 loop
        /*Obtención de un DNI*/
        v_dni := fun.crear_dni;
        /*Comprobación de que no existe un cliente con el dni
        generado*/
        select count(*)
        into v_existe_dni
        from cliente
        where dni=v_dni;
        if v_existe_dni =0
        then
            insert into cliente (
                DNI,
                NOMBRE,
                EMAIL
            )
            values (
                v_dni,
                fun.crear_nombre,
                fun.crear_email
            );
        end if;
    end loop;
    commit;
end;
```

3.3.4 Carga de la tabla AGENDA

La carga de esta tabla es sencilla, características:

- Para cada cliente se va a generar un número aleatorio entre 1 y 3 y se le insertará esa cantidad de teléfonos.
- Columnas:
 - *dni*: clave ajena a la tabla Cliente.
 - *telefono*: teléfono generado con la función *fun.crear_telefono*.

```

declare
/*Obtención de todas las filas de la tabla Cliente*/
    cursor cliente_cur is select * from cliente;
    v_cant_tel number;
    v_telefono agenda.telefono%type;
    v_existe number;
begin
    for cliente_fila in cliente_cur loop
/*Generación de un número aleatorio entre 1 y 3 para saber cuántos
teléfonos se le van a insertar*/
        v_cant_tel:=trunc(dbms_random.value(1,4));
        for i in 1..v_cant_tel loop
            v_telefono:=fun.crear_telefono;
            select count(*)
                into v_existe
                from agenda
                where dni = cliente_fila.dni
                and telefono = v_telefono;
            if v_existe=0 then
                insert into agenda (
                    DNI,
                    TELEFONO
                )
                values (
                    cliente_fila.dni,
                    v_telefono
                );
            end if;
        end loop;
    end loop;
    commit;
end;
```

3.3.5 Carga de la tabla VARIEDAD_CAFÉ

La carga de esta tabla es sencilla, características:

- Número de filas 20: el bucle va hasta 20.
- Columnas:
 - *cod*: alfanumérico creado con crear_cod_variedad.
 - *denominación*: alfanumérica creado con generar_denominación.
 - *PVP*: real, creado con la función crear_pvp.
 - *intensidad*: entero entre 1 y 12, creado con la función crear_intensidad.
 - *nivel_cafeina*: alfanumérico, generado con crear_nivel_cafeina

```

begin
    for i in 1..20 loop
```

```

insert into variedad_cafe (
    cod,
    denominacion,
    pvp,
    intensidad,
    nivel_cafeina
)
values (
    crear_cod_variedad,
    crear_denominacion,
    crear_pvp,
    crear_intensidad,
    crear_nivel_cafeina
);
end loop;
commit;
end;

```

3.3.6 Carga de la tabla OFERTA

La carga de esta tabla es sencilla, características:

- Número de filas 30: el bucle va hasta 30.
- Columnas:
 - *anio*: NUMBER. Generado con la función crear_anio
 - *semana*: NUMBER. Entero entre 1 y 52 creado con la función crear_semana
 - *fecha_fin*: DATE. Se genera con la función crear_fecha.
 - *descuento*: NUMBER. Valor entero entre 1 y 100 creado con la función crear_descuento.
 - *cod*: Clave ajena de la tabla Variedad_café.

```

declare
    v_cod          variedad_cafe.cod%type;
    v_cuantos      number;
    v_random       number;
    v_anio         number;
    v_fecha_fin    date;
begin
    /* número total de variedades de café */
    select count(*)
    into v_cuantos
    from variedad_cafe;

    for i in 1..30 loop

        /* selección aleatoria de una variedad existente */
        v_random := trunc(dbms_random.value(0, v_cuantos));
    end loop;
end;

```

```
select cod
into v_cod
from variedad_cafe
offset v_random rows
fetch next 1 rows only;

/* generar año */
v_anio := crear_anio();

/* generar fecha dentro de ese año */
v_fecha_fin :=
  to_date('01/01/' || v_anio, 'dd/mm/yyyy')
  + trunc(dbms_random.value(0, 365));

insert into oferta (
  anio,
  semana,
  fecha_fin,
  descuento,
  cod
) values (
  v_anio,
  semana(),
  v_fecha_fin,
  crear_descuento(),
  v_cod
);

end loop;

commit;
end;
```

3.3.7 Carga de la tabla REGIÓN

- Número de filas:
Se insertan 30 regiones por cada país existente, mediante un bucle que va de 1 a 30 para cada país.
- Columnas:
 - *cod_país*: clave ajena a la tabla *PAIS*, obtenida recorriendo dicha tabla, garantizando así el cumplimiento de la clave foránea.
 - *cod_region*: se genera mediante la función *crear_cod_region*, produciendo un código alfanumérico único para cada región.
 - *nombre_region*: Se genera de forma secuencial mediante concatenación de 'Región_' y un contador desde el '01' al '30'.

```
declare
  cursor pais_cursor is
    select cod_pais from pais;

  v_cod_pais      region.cod_pais%type;
  v_nombre_region region.nombre_region%type;
begin
  for v_pais in pais_cursor loop
    for i in 1..30 loop

      v_cod_pais      := v_pais.cod_pais;

      if i <= 9 then
        v_nombre_region := 'region_0' || i;
      else
        v_nombre_region := 'region_' || i;
      end if;

      insert into region (
        cod_pais,
        cod_region,
        nombre_region
      ) values (
        v_cod_pais,
        crear_cod_region,
        v_nombre_region
      );

    end loop;
  end loop;

  commit;
end;
```

3.3.8 Carga de la tabla SE_PRODUCIR

- Número de filas:
El número de filas depende de las combinaciones entre granos y regiones, generándose asociaciones válidas entre ambas tablas mediante bucles.
- Columnas:
 - *cod_grano*; clave ajena a la tabla *GRANO*, obtenida recorriendo dicha tabla.
 - *cod_pais*; clave ajena a la tabla *PAIS*, obtenida a partir de la tabla *REGION*.
 - *cod_region*; clave ajena a la tabla *REGION*.

```
declare

cursor grano_cursor is
    select cod_grano from grano;

cursor region_cursor is
    select cod_pais, cod_region from region;

v_cod_grano    se_produce.cod_grano%type;
v_cod_pais     se_produce.cod_pais%type;
v_cod_region   se_produce.cod_region%type;
begin
    for v_grano in grano_cursor loop
        for v_region in region_cursor loop

            v_cod_grano    := v_grano.cod_grano;
            v_cod_pais     := v_region.cod_pais;
            v_cod_region   := v_region.cod_region;

            insert into se_produce (
                cod_grano,
                cod_pais,
                cod_region
            ) values (
                v_cod_grano,
                v_cod_pais,
                v_cod_region
            );

        end loop;
    end loop;

    commit;
end;
```

3.3.9 Carga de la tabla MEZCLA

- **Número de filas:**
El número de filas depende del número de variedades de café existentes y del número de granos seleccionados para cada una de ellas. Para cada variedad se generan entre 2 y 5 filas, correspondientes a los distintos granos que forman parte de su mezcla.
- **Columnas:**
 - *cod*: clave ajena a la tabla *VARIEDAD_CAFE*, obtenida recorriendo dicha tabla mediante un cursor.
 - *cod_grano*: clave ajena a la tabla *GRANO*, seleccionada de forma aleatoria entre los granos existentes.
 - *cantidad_grano*: valor numérico real que representa la cantidad de ese grano por cada 100 gramos de variedad. Las cantidades se generan de forma que la suma total por variedad sea exactamente 100, cumpliendo la restricción CHECK definida en la tabla.

- *nivel_molido*: valor entero comprendido entre 0 y 5, generado mediante la función *crear_nivel_molido*, de acuerdo con las restricciones de integridad.

```
declare
  cursor variedad_cursor is
    select cod from variedad_cafe;

  v_num_granos number;
  v_restante   number;

  v_cod_grano  grano.cod_grano%type;
  v_cantidad   mezcla.cantidad_grano%type;
  v_nivel      mezcla.nivel_molido%type;

begin
  for v_variedad in variedad_cursor loop

    -- Número aleatorio de granos por variedad (entre 2 y 5)
    v_num_granos := trunc(dbms_random.value(2, 6));
    v_restante   := 100;

    for i in 1..v_num_granos loop
      select cod_grano
      into v_cod_grano
      from (
        select cod_grano
        from grano
        order by dbms_random.value
      )
      where rownum = 1;

      if i = v_num_granos then
        v_cantidad := v_restante;
      else
        v_cantidad := round(
          dbms_random.value(1, v_restante - (v_num_granos - i)),
          2
        );
      end if;

      v_restante := v_restante - v_cantidad;
      v_nivel    := crear_nivel_molido;

      insert into mezcla (
        cod,
        cod_grano,
        cantidad_grano,
        nivel_molido
      )
      select
        v_variedad.cod,
        v_cod_grano,
```

```

        v_cantidad,
        v_nivel
    from dual
    where not exists (
        select 1
        from mezcla
        where cod = v_variedad.cod
            and cod_grano = v_cod_grano
    );

    end loop;
end loop;

commit;
end;
```

3.3.10 Carga de la tabla VARIEDAD_ESPECIAL

- Número de filas: No todas las variedades de café son especiales; la inserción se realiza de forma condicionada.
- Columnas:
 - *Cod*: Clave ajena a la tabla *VARIEDAD_CAFE*, obtenida recorriendo dicha tabla.
 - *Descripción_envase*: Generada mediante la función *crear_descripcion_envase*, produciendo una cadena aleatoria de hasta 100 caracteres.
 - *Fecha_ini*: Generada mediante la función *crear_fecha*.
 - *Fecha_fin*: Generada mediante la función *crear_fecha_fin*, garantizando que sea mayor o igual que la fecha de inicio.

```

declare
    cursor variedad_cursor is
        select cod from variedad_cafe;

    v_fecha_ini          variedad_especial.fecha_ini%type;
    v_fecha_fin          variedad_especial.fecha_fin%type;
begin
    for v_variedad in variedad_cursor loop

        if trunc(dbms_random.value(0,2)) = 1 then
            v_fecha_ini          := crear_fecha;
            v_fecha_fin          := crear_fecha_fin(v_fecha_ini);

            insert into variedad_especial (
                cod,
                descripcion_envase,
                fecha_ini,
                fecha_fin
            ) values (
```

```

        v_variedad.cod,
        crear_descripcion_envase,
        v_fecha_ini,
        v_fecha_fin
    );
    end if;
end loop;

commit;
end;
```

3.3.11 Carga de la tabla PAÍS

- Número de filas: 20 países.
- Columnas:
 - *cod_pais*: Código alfanumérico único de 5 caracteres (generado con *crear_cod_pais*).
 - *Nombre_pais*: Nombre alfanumérico de 10 caracteres (generado con *crear_nombre*).

```

begin
    for i in 1..20 loop
        insert into Pais (
            cod_pais,
            nombre_pais
        ) values (
            crear_cod_pais,
            crear_nombre
        );
    end loop;
    commit;
end;
```

3.3.12 Carga de la tabla GRANO

- Número de filas: 30 tipos de grano.
- Columnas:
 - *cod_grano*: Código alfanumérico único (generado con *crear_cod_grano*)
 - *descripción_grano*: descripción creada con *crear_descripción_grano*

```

begin
    for i in 1..30 loop
        insert into Grano (
            cod_grano,
            descripcion_grano
        ) values (
            crear_cod_grano,
            crear_descripcion_grano
        );
    end loop;
```

```
commit;
end;
```

3.3.13 Carga de la tabla NATURAL

- Número de filas: Solo algunos aditivos serán naturales.
- Se recorren los aditivos existentes (y no sé cómo seleccionar cuales sean los naturales y cuales los artificiales)
- Columnas:
 - *cod_aditivo*: Clave ajena a Aditivo
 - *nombre_comun*: Nombre común generado con la función *crear_nombre*

```
declare
  cursor aditivo_cursor is select cod_aditivo
                             from Aditivo
                             where cod_aditivo not in (
                               select cod_aditivo
                               from Artificial
                             );
  v_cod_aditivo VARCHAR2(10);
  v_nombre_comun VARCHAR2(10);
begin
  for aditivo in aditivo_cursor loop
    v_cod_aditivo := aditivo.cod_aditivo;
    v_nombre_comun := crear_nombre();
    insert into "NATURAL" (
      cod_aditivo,
      nombre_comun
    ) values (
      v_cod_aditivo,
      v_nombre_comun
    );
  end loop;
  commit;
end;
```

3.3.14 Carga de la tabla INCLUYE

- Número de filas: Relación entre variedades de café y aditivos
- Columnas:
 - *cod_aditivo*: Clave ajena a Aditivo, obtenida recorriendo dicha tabla.
 - *cod*: Clave ajena a Variedad_cafe, obtenida recorriendo dicha tabla.
 - *cantidad_aditivo*: Cantidad entre 1 y 100 generada con *crear_cantidad_aditivo*

```
declare
  cursor variedad_cursor is
    select cod from variedad_cafe;

  v_cod          variedad_cafe.cod%type;
```

```
v_cod_aditivo    aditivo.cod_aditivo%type;
v_num_aditivos   number;
begin
  for variedad in variedad_cursor loop
    v_cod := variedad.cod;
    v_num_aditivos := trunc(dbms_random.value(0,6));

    for i in 1..v_num_aditivos loop

      select cod_aditivo
      into v_cod_aditivo
      from (
        select cod_aditivo
        from aditivo
        order by dbms_random.value
      )
      where rownum = 1;

      insert into incluye (
        cod_aditivo,
        cod,
        cantidad_aditivo
      )
      select
        v_cod_aditivo,
        v_cod,
        crear_cantidad_aditivo()
      from dual
      where not exists (
        select 1
        from incluye
        where cod_aditivo = v_cod_aditivo
          and cod = v_cod
      );

    end loop;
  end loop;

  commit;
end;
```

3.3.15 Carga de la tabla ADITIVO

La carga de esta tabla es sencilla, características:

- Número de filas: 50
- Columnas:
 - *cod_aditivo*: alfanumérico único, generado con la función crear_cod_aditivo.

- `descripcion_aditivo`: descripción aleatoria de hasta 100 caracteres, generada con la función `crear_descripcion_aditivo`.

```
begin
  for i in 1..50 loop
    insert into aditivo (
      cod_aditivo,
      descripcion_aditivo
    ) values (
      crear_cod_aditivo,
      crear_descripcion_aditivo
    );
  end loop;
  commit;
end;
```

3.3.16 Carga de la tabla ARTIFICIAL

La carga de esta tabla depende de los aditivos existentes:

- Número de filas: Variable, depende de cuántos aditivos se clasifiquen como artificiales.
- Columnas:
 - `cod_aditivo`: clave ajena a Aditivo.
 - `cod_eu`: código único europeo generado con la función `crear_cod_eu`.
- Se asigna de forma aleatoria si un aditivo será artificial (`dbms_random.value`).

```
declare
  cursor aditivo_cursor is
    select cod_aditivo from aditivo;
  v_cod_aditivo aditivo.cod_aditivo%type;

begin
  for aditivo in aditivo_cursor loop
    if trunc(dbms_random.value(0,2)) = 1 then
      v_cod_aditivo := aditivo.cod_aditivo;

      insert into artificial (
        cod_aditivo,
        cod_eu
      ) values (
        v_cod_aditivo,
        crear_cod_eu
      );
    end if;
  end loop;
  commit;
end;
```

3.3.17 Carga de la tabla COMPRA

La tabla COMPRA guarda las compras realizadas por los clientes.

- Número de filas: Igual al número de clientes existentes.
- Columnas:
 - *num_sec*: número secuencial de la compra, empezando desde 1.
 - *fecha*: fecha aleatoria de la compra, generada con la función *crear_fecha*.
 - *importe*: inicialmente 0, se recalcula luego con la función *calcular_importe*.
 - *tarjeta*: indica si se usó tarjeta, generado con la función *tarjeta*.
 - *DNI*: clave ajena a Cliente.

```
declare
  cursor cliente_cursor is
    select dni from cliente;
begin
  for cliente in cliente_cursor loop
    insert into compra (
      num_sec,
      fecha,
      importe,
      tarjeta,
      dni
    ) values (
      crear_num_compra,
      crear_fecha,
      0,
      tarjeta,
      cliente.dni
    );

  end loop;
  commit;
end;
```

3.3.18 Carga de la tabla COMPRA_VARIEDAD

Relaciona compras con variedades de café.

- Número de filas: Igual al número de compras (una por cada compra, aunque podría ampliarse si se permite más de una variedad por compra).
- Columnas:
 - *num_sec*: clave ajena a Compra.
 - *cod*: clave ajena a Variedad_cafe.
 - *cantidad_compra*: cantidad comprada, generada con la función *crear_cantidad_compra*.
- Se selecciona la variedad de café de forma aleatoria con *dbms_random.value*.

```
declare
  cursor compra_cursor is
    select num_sec from compra;
  v_cod variedad_cafe.cod%type;
begin
```

```
for compra in compra_cursor loop
  select cod
  into v_cod
  from (
    select cod
    from variedad_cafe
    order by dbms_random.value
  )
  where rownum = 1;

  insert into compra_variedad (
    num_sec,
    cod,
    cantidad_compra
  ) values (
    compra.num_sec,
    v_cod,
    crear_cantidad_compra
  );
end loop;
commit;
end;
```

3.3.19 Recalcular el importe de las compras

Después de cargar COMPRA y COMPRA_VARIEDAD, se actualiza el importe total de cada compra.

- Número de filas afectadas: Todas las compras existentes.
- Columnas afectadas:
 - *importe*: se actualiza sumando el coste de todas las variedades asociadas a la compra mediante `calcular_importe(num_sec)`.

```
begin
  for c in (select num_sec from compra) loop
    update compra
    set importe = calcular_importe(c.num_sec)
    where num_sec = c.num_sec;
  end loop;
  commit;
end;
```

4. TAREA 4: INFORMES Y PROGRAMACIÓN

Tarea	Contenido	Puntos	Trabajo a realizar	Contenido de la entrega
4	Informes	0,75 puntos	Diseñar al menos 6 informes de complejidad variada usando en BIRT (para realizar esta tarea deben haberse cargado datos en la base de datos de Oracle).	La entrega final consta de dos partes: - Memoria: un solo documento pdf "de calidad" que incluye: - Todas las entregas realizadas en las tareas anteriores con las correcciones hechas por el profesor. - Descripción de los informes generados y pantallazo de todo o de parte del cada informe. - Descripción y código de los módulos programados. - Un fichero <i>.rptdesign</i> por cada informe diseñado.
	Módulos programados	0,75 puntos	Con el usuario Oracle del grupo de trabajo, escribir al menos: 3 procedimientos 3 funciones 3 disparadores El código debe ser original, no se aceptan copias de "Alquiler de Vehículos"	
	Memoria final	0,25 punto	Redactar y maquetar documento final	

4.1 Informes

- Listado de todas las variedades de café mostrando sus características (denominación, PVP, intensidad y nivel de cafeína), ordenadas de forma ascendente según el precio de venta al público (PVP) y agrupadas según si han sido una variedad especial o no.
- Listado de las ofertas asociadas a cada variedad de café, mostrando el descuento aplicado, el año, la semana y la fecha de finalización de cada oferta.
- Listado de las variedades de café agrupadas por nivel de cafeína, indicando el número de variedades en cada nivel, acompañado de un gráfico de barras representativo.
- Listar las compras más grandes y las compras más pequeñas obteniendo el nombre de los clientes que las han realizado, agrupando las compras según si han usado tarjeta o no, así como realizar dos gráficos de las compras anteriormente explicadas respectivamente.
- Dado un código de país como parámetro de entrada, listar los granos que se producen en dicho país y calcular el número de regiones, tanto pertenecientes a ese país como a otros, en las que se produce cada grano.
- Dado un número de secuencia de una compra como parámetro, mostrar los datos de la compra, el nombre del cliente, su correo electrónico y todos los números de teléfono asociados al cliente.

4.1.1 Listado 1

Informe de las variedades de café según su especialidad

Denominación	Intensidad	Nivel de cafeína	PVP
Especial			
variedad_09	1	alto	1,79€
variedad_18	7	bajo	5,67€
variedad_15	1	alto	8,46€
variedad_12	12	medio	9,58€
variedad_20	2	bajo	10,33€
variedad_06	10	bajo	11,65€
variedad_14	5	medio	13,27€
variedad_13	9	sin	13,96€
variedad_01	6	medio	14,06€
variedad_11	8	sin	14,62€
variedad_02	11	sin	15,79€
variedad_05	7	medio	18,40€
variedad_07	6	bajo	18,65€
No especial			
variedad_19	9	medio	6,13€
variedad_08	2	bajo	7,24€
variedad_10	7	sin	8,28€
variedad_17	3	medio	10,50€
variedad_04	3	bajo	12,17€
variedad_16	5	medio	15,50€
variedad_03	10	alto	16,88€

4.1.2 Listado 2

Informe de las ofertas aplicadas a cada variedad de café

	Año	Fecha de finalización	Descuento
variedad_01	PVP: 14,06 €	Intensidad: 6	Nivel de cafeína: medio
	2022	3 de marzo de 2022	83%

Descuento medio de la variedad: 83,00%			
Total de las ofertas de la variedad: 1			
variedad_02	PVP: 15,79 €	Intensidad: 11	Nivel de cafeína: sin
	2010	13 de noviembre de 2010	42%

Descuento medio de la variedad: 42,00%			
Total de las ofertas de la variedad: 1			
variedad_04	PVP: 12,17 €	Intensidad: 3	Nivel de cafeína: bajo
	2003	27 de abril de 2003	51%
	2006	15 de noviembre de 2006	27%
	2017	23 de marzo de 2017	52%

Descuento medio de la variedad: 43,33%			
Total de las ofertas de la variedad: 3			
variedad_06	PVP: 11,65 €	Intensidad: 10	Nivel de cafeína: bajo
	2004	31 de mayo de 2004	22%
	2006	13 de abril de 2006	79%
	2013	2 de noviembre de 2013	48%

Descuento medio de la variedad: 49,67%			
Total de las ofertas de la variedad: 3			
variedad_07	PVP: 18,65 €	Intensidad: 6	Nivel de cafeína: bajo
	2005	9 de junio de 2005	67%
	2016	25 de enero de 2016	49%

Descuento medio de la variedad: 58,00%			
Total de las ofertas de la variedad: 2			
variedad_08	PVP: 7,24 €	Intensidad: 2	Nivel de cafeína: bajo
	2024	25 de septiembre de 2024	5%

Descuento medio de la variedad: 5,00%			
Total de las ofertas de la variedad: 1			
variedad_10	PVP: 8,28 €	Intensidad: 7	Nivel de cafeína: sin
	2010	1 de agosto de 2010	6%

Descuento medio de la variedad: 6,00%			
Total de las ofertas de la variedad: 1			
variedad_11	PVP: 14,62 €	Intensidad: 8	Nivel de cafeína: sin
	2004	22 de mayo de 2004	40%
	2020	15 de junio de 2020	87%

Descuento medio de la variedad: 63,50%			
Total de las ofertas de la variedad: 2			
variedad_13	PVP: 13,96 €	Intensidad: 9	Nivel de cafeína: sin
	2022	28 de enero de 2022	41%

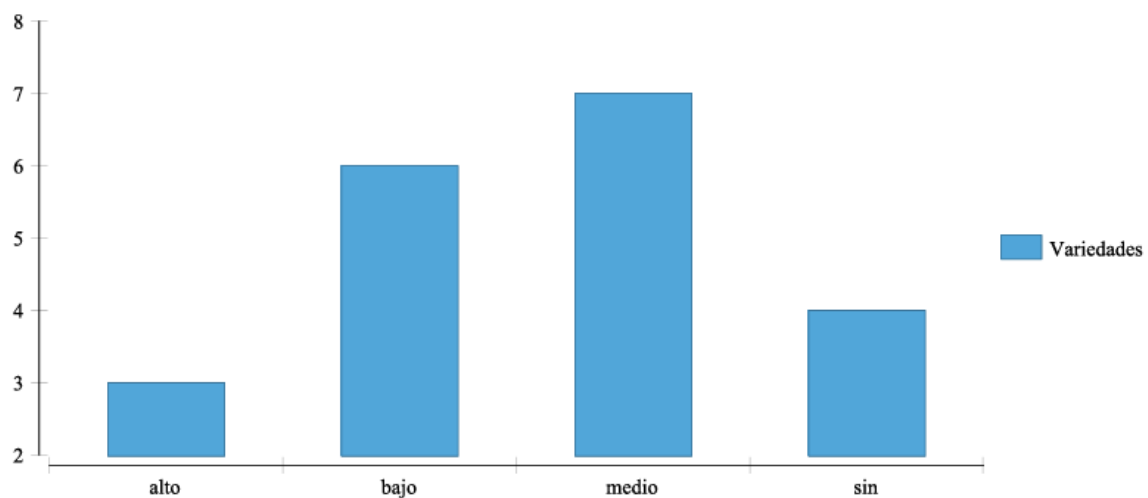
Descuento medio de la variedad: 41,00%			
Total de las ofertas de la variedad: 1			
variedad_14	PVP: 13,27 €	Intensidad: 5	Nivel de cafeína: medio
	2015	9 de junio de 2015	67%
	2017	31 de mayo de 2017	59%

Descuento medio de la variedad: 63,00%			
Total de las ofertas de la variedad: 2			
variedad_15	PVP: 8,46 €	Intensidad: 1	Nivel de cafeína: alto
	2003	22 de agosto de 2003	89%
	2005	30 de agosto de 2005	95%

4.1.3 Listado 3

Informe de las variedades de café según el nivel de cafeína

Denominación	Intensidad	PVP
Nivel cafeína: alto		
Total de variedades: 3		
variedad_09	1	1,79€
variedad_15	1	8,46€
variedad_03	10	16,88€
Nivel cafeína: bajo		
Total de variedades: 6		
variedad_18	7	5,67€
variedad_08	2	7,24€
variedad_20	2	10,33€
variedad_06	10	11,65€
variedad_04	3	12,17€
variedad_07	6	18,65€
Nivel cafeína: medio		
Total de variedades: 7		
variedad_19	9	6,13€
variedad_12	12	9,58€
variedad_17	3	10,50€
variedad_14	5	13,27€
variedad_01	6	14,06€
variedad_16	5	15,50€
variedad_05	7	18,40€
Nivel cafeína: sin		
Total de variedades: 4		
variedad_10	7	8,28€
variedad_13	9	13,96€
variedad_11	8	14,62€
variedad_02	11	15,79€

Informe de las variedades de café según el nivel de cafeína**Cantidad de variedades de café según el nivel de cafeína**

4.1.4 Listado 4

Compras más grandes

CLIENTE	FECHA
IMPORTE: 186,5	
USA TARJETA N	
Muñoz Jiménez, Elena	5/7/01
Baroja Belaib, Micaela	18/5/04
Vázquez Vicent, Salma	27/5/06
Mota Romero, Omar	8/12/09
Markessinis Almeida, Amanda	10/3/11
Herranz Sánchez, Ramón	25/11/11
Llosa Hurtado, Lorena	10/12/11
Moreno Ramos, Avelina	13/5/13
González Tortosa, Daniela	15/7/13
Segarra Díaz, Alejandro	8/11/13
Onrrubia Ruiz, Ricardo	12/12/13
Ródenas Bilbao, Juan	27/5/17
Molinero Navarro, Nora	16/12/17
Jiménez Carratalá, Evelio	7/7/18
Marqués Bilbao, Ramón	1/12/19
Márquez Almeida, Lucas	12/8/20
Alonso Vázquez, Teodomira	30/10/20
Ruiz Tortosa, Benigno	5/5/21
Martínez Márquez, Hugo	24/2/22
Belaib Cervantes, Leocadio	19/5/22
USA TARJETA S	
López Herranz, Lucas	5/2/02
Verde Ródenas, Irene	25/6/03
Regadera Herranz, Martina	28/6/03
Mota Carratalá, Enrique	12/5/04
Reverte Ramos, Lucía	8/6/05
Markessinis Cervantes, Daniel	28/10/05
Romero López, Mateo	23/4/06
Vázquez Hernández, Mateo	8/6/06
Reverte Ródenas, Daniela	29/1/09
Muñoz Llosa, Lorena	19/6/10
Hernández Sanchis, Carmen	6/12/10
Reverte Romero, Leonor	24/5/11
Herranz Bilbao, Elisa	18/11/11
Ramos Markessinis, Leonor	16/10/12
Martínez González, Evelio	6/2/15
Muñoz Gómez, Lorena	13/2/17
Martínez Rodríguez, Elena	14/4/17
Vázquez Segarra, Julia	26/7/17
Ruiz Sánchez, Elena	26/8/17
Gutiérrez Sánchez, Ricardo	18/2/19

Compras más pequeñas

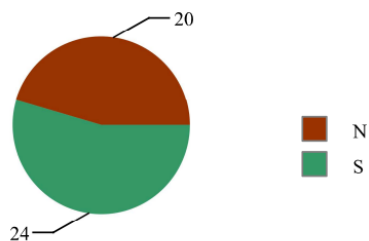
CLIENTE	FECHA
IMPORTE: 1,79	
USA TARJETA N	
Onrrubia Muñoz, Julia	11/2/01
Muñoz Segarra, Manuel	2/2/03
Aliaga Casamayor, Alba	20/2/03
Cervantes Segarra, Amanda	25/3/03
Markessinis Muñoz, Alicia	15/5/05
Llosa Navarro, Avelina	18/3/06
Marqués Markessinis, Agapito	11/5/07
Aliaga Belaib, Hugo	17/2/09
Santillana Ródenas, Paula	30/7/09
Moreno Segarra, Vicente	1/8/09
Regadera Aliaga, Alicia	27/11/09
López Gutiérrez, Trinidad	20/5/10
Aliaga Molinero, Alicia	2/7/11
Segarra Álvarez, Lucas	12/10/12
Vargas Onrrubia, Julia	12/5/13
López Pérez, Micaela	25/7/14
Almeida Ramos, Omar	14/7/15
Hernández Hernández, Laura	25/8/16
García Belaib, Paula	31/8/16
Reverte Baroja, María	19/9/16
Verde Belaib, Manuel	21/6/17
Ruiz Carratalá, Enrique	1/7/17
Bilbao Ruiz, Alejandro	1/7/18
Mota Bilbao, Leonor	5/8/18
Peleato Navarro, Nora	15/3/19
Vicent Ramos, Aisha	20/12/20
González Torres, Alicia	25/4/22
Marqués Baroja, Daniela	19/8/23
Pérez Álvarez, Carmen	6/5/24
Tortosa Márquez, Rubén	5/6/24
González Santamans, Leo	23/8/25
USA TARJETA S	
Santillana Reverte, Julia	15/3/02
Santamans Vicent, Lucas	12/6/02
Segarra Segarra, Avelina	3/8/02
Ramos Gómez, Julia	9/8/03
Segarra Rodríguez, Salma	21/2/04
Sanchis Santillana, Zoe	5/9/06
Jiménez González, Irene	30/3/08
Grandes Sánchez, Amanda	2/4/08
Vargas Muñoz, María	8/2/09

19 ene 2026, 0:10

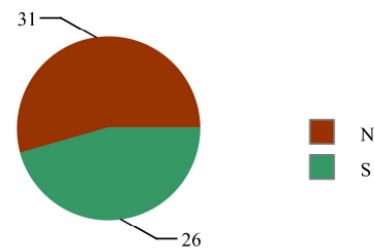
CLIENTE	FECHA
IMPORTE: 186,5	
USA TARJETA S	
Marqués Vicent, Manuel	1/9/19
Martínez Gutiérrez, Amanda	15/1/22
Llosa Herranz, Julia	26/6/22
Grandes Ródenas, Laura	3/8/25

CLIENTE	FECHA
IMPORTE: 1,79	
USA TARJETA S	
Ródenas Jiménez, Paula	13/12/09
Vicent Vázquez, Lorena	18/12/09
Alonso Molinero, Carmen	27/6/10
Navarro Casamayor, Emma	11/1/11
Torres Hurtado, Leocadio	20/8/11
Gutiérrez Gómez, Álvaro	5/10/11
López Grandes, Trinidad	22/2/13
Martínez Casamayor, Trinidad	22/7/13
Regadera Domínguez, Leocadio	3/12/14
Sanchis Pérez, Julia	10/4/17
Martín Belaib, José	25/7/17
Pérez Martínez, Martina	21/1/20
López Regadera, César	19/2/21
Herranz Hernández, Rubén	24/8/22
Tortosa Herranz, Martina	14/11/22
Alonso Onrrubia, Irina	18/11/22
Baroja Markessinis, Irina	13/12/25

Compran con tarjeta (S) o no (N) en compras grandes

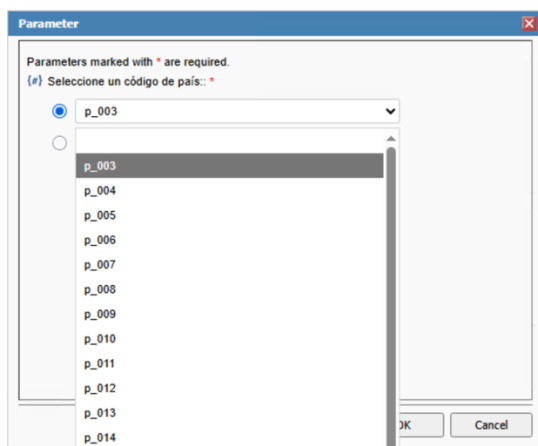


Compran con tarjeta (S) o no (N) en compras pequeñas



4.1.5 Listado 5

Se ha creado un parámetro de tipo “combo” para que el usuario seleccione el código de país y ver en cuántas regiones de ese país se produce cada tipo de grano.

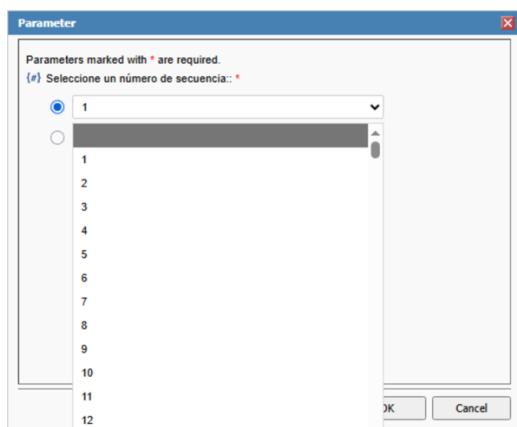


Informe de los granos de café se producen en el país : p_003

Código de grano	Descripción del grano	Nº de regiones en las que se produce dentro del país	Nº de regiones en las que se produce fuera del país
grano_001	WrZHOSHBAblxwCTUyNyqktHcjwgnNAeyTznRxZhxFNJDf	2	0
grano_002	MEaOpMFsZRcRFAdyKEaIeJKPiiMlhrXRyZkSMKtEOgYMj	1	3
grano_005	dBgRYiwcjcnCHMgaQNQvOwyFfBsNWPOhLMUgbHiqITC\	1	2
grano_016	uAXtUAUOMvgvjIBuLxxVVIghwLbWVFauZzSwbnDDIah	2	3
grano_022	qMzLcXvaCppNiobhAbQaqpNcixLyhDWAMEBtBLFKpGQR	2	3
grano_025	CaWStZMBuLahncmqewFkNrpZSTxhKtOoNYmugSSVUKV\	2	3
grano_029	wBbkNjniLlmRQfUXwQVPDguLtGODrhUbhBQuIgdVhLAR	2	3

4.1.6 Listado 6

Se ha creado un parámetro de tipo “combo” para que el usuario elija el número secuencial de la compra de la que quiere obtener el contacto del cliente.

**Listado de los teléfonos de contacto de un comprador**

Número secuencial de la compra: 238
 Fecha de la compra: 28 de agosto de 2004
 Importe de la compra: 41,88 €
 Pago en tarjeta: S

DNI del cliente: 12008414A
 Nombre del cliente: Navarro Jiménez, Emma
 Email del cliente: kdveb@pds.ses.yz

Teléfonos de contacto:

204897463
 537278275
 968504601

4.2 Programación

4.2.1 Funciones

Función importe_compra

Esta función calcula el importe total de una compra a partir de las variedades incluidas en ella. Si la compra no existe, la función devuelve cero.

```
create or replace function importe_compra (
    p_num_compra compra.num_sec%type
) return number is
    v_importe number;
    compra_no_existe exception;
begin
    select sum(cv.cantidad_compra * v.pvp)
    into v_importe
    from compra_variedad cv
    join variedad_cafe v
    on v.cod = cv.cod
    where cv.num_sec = p_num_compra;

    if v_importe is null then
        raise compra_no_existe;
    end if;

    return v_importe;

exception
    when compra_no_existe then
        return 0;
end;
```

La función se puede invocar usando la tabla ficticia dual:

```
select importe_compra(1) from dual;
```

Función num_variedades_compra

Esta función devuelve el número de variedades distintas incluidas en una compra.

```
create or replace function num_variedades_compra (
    p_num_compra compra.num_sec%type
) return number is
    v_total number;
begin
    select count(*)
    into v_total
    from compra_variedad
```

```

        where num_sec = p_num_compra;

        return v_total;
end;
```

La función se puede invocar usando la tabla ficticia dual:

```
select num_variedades_compra(1) from dual;
```

Función intensidad_media_compra

Esta función calcula la intensidad media del café de una compra, teniendo en cuenta la intensidad de cada variedad incluida.

```

create or replace function intensidad_media_compra (
    p_num_compra compra.num_sec%type
) return number is
    v_media number;
begin
    select avg(v.intensidad)
    into v_media
    from compra_variedad cv
    join variedad_cafe v
        on v.cod = cv.cod
    where cv.num_sec = p_num_compra;

    return v_media;
end;
```

La función se puede invocar usando la tabla ficticia dual:

```
select intensidad_media_compra(1) from dual;
```

4.2.2 Procedimientos

Procedimiento registrar_compra

Este procedimiento crea una nueva compra para un cliente determinado, inicializando el importe a cero. El importe se actualizará automáticamente mediante los disparadores definidos anteriormente.

```

create or replace procedure registrar_compra (
    p_dni cliente.dni%type,
    p_pago_tarjeta compra.tarjeta%type
) is
    v_num_compra compra.num_sec%type;
begin
    select seq_num_compra.nextval
    into v_num_compra
```

```
from dual;

insert into compra (
    num_sec,
    fecha,
    importe,
    tarjeta,
    dni
)
values (
    v_num_compra,
    sysdate,
    0,
    p_pago_tarjeta,
    p_dni
);

dbms_output.put_line(
    'compra registrada con numero ' || v_num_compra
);
end;
```

Procedimiento crear_oferta

Este procedimiento permite registrar una nueva oferta semanal para una variedad de café, almacenando el histórico de ofertas realizadas.

```
create or replace procedure crear_oferta (
    p_cod_variedad variedad_cafe.cod%type,
    p_anio number,
    p_semana number,
    p_descuento number
) is
begin
    insert into oferta (
        cod,
        anio,
        semana,
        descuento,
        fecha_fin
    )
    values (
        p_cod_variedad,
        p_anio,
        p_semana,
        p_descuento,
        sysdate + 7
    );
end;
```

```
        dbms_output.put_line('oferta creada correctamente');
end;
```

Procedimiento borrar_clientes_sin_compras

Este procedimiento elimina de la base de datos aquellos clientes que no han realizado ninguna compra. No se incluye la sentencia commit para poder deshacer la operación si se desea.

```
create or replace procedure borrar_clientes_sin_compras is
begin
    delete from cliente
    where dni not in (
        select dni
        from compra
        where dni is not null
    );

    dbms_output.put_line(
        'clientes borrados: ' || sql%rowcount
    );
end;
```

4.2.3 Disparadores

Mantenimiento del importe de una compra (inserción)

Para que el atributo derivado importe de la tabla compra tenga siempre el valor correcto, se define un disparador que recalcula dicho importe cada vez que se inserta una nueva variedad en una compra.

```
create or replace trigger ins_compra_variedad
after insert on compra_variedad
for each row
begin
    update compra c
    set importe = (
        select sum(cv.cantidad_compra * v.pvp)
        from compra_variedad cv
        join variedad_cafe v
        on v.cod = cv.cod
        where cv.num_sec = :new.num_sec
    )
    where c.num_sec = :new.num_sec;
end;
```

Mantenimiento del importe de una compra (modificación)

Este disparador actualiza el importe de la compra cuando se modifica la cantidad de cajas de una variedad incluida en la compra.

```
create or replace trigger upd_compra_variedad
after update of cantidad_compra on compra_variedad
for each row
begin
    update compra c
    set importe = (
        select sum(cv.cantidad_compra * v.pvp)
        from compra_variedad cv
        join variedad_cafe v
        on v.cod = cv.cod
        where cv.num_sec = :new.num_sec
    )
    where c.num_sec = :new.num_sec;
end;
```

Control de una restricción de integridad

Una compra debe contener al menos una variedad de café. Este disparador impide que se elimine la última variedad asociada a una compra.

```
create or replace trigger del_ultima_variedad
before delete on compra_variedad
for each row
declare
    v_total number;
begin
    select count(*)
    into v_total
    from compra_variedad
    where num_sec = :old.num_sec;

    if v_total = 1 then
        raise_application_error(
            -20001,
            'una compra debe contener al menos una variedad de cafe'
        );
    end if;
end;
```

Mantenimiento del importe de una compra (borrado)

Este disparador actualiza el importe total de una compra cuando se elimina una variedad incluida en ella, manteniendo la coherencia del atributo derivado *importe* mientras la compra se encuentra en proceso de registro.

```
create or replace trigger del_compra_variedad
after delete on compra_variedad
for each row
begin
    update compra c
    set importe = (
        select nvl(sum(cv.cantidad_compra * v.pvp),0)
        from compra_variedad cv
        join variedad_cafe v
        on v.cod = cv.cod
        where cv.num_sec = :old.num_sec
    )
    where c.num_sec = :old.num_sec;
end;
```